



Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Análisis comparativo de modelos para la detección de audios deepfake

Comparative Analysis of Models for Deepfake Audio Detection

Samuel Pérez López

La Laguna, 15 de julio de 2026

Dña. **Pino Caballero Gil**, Catedrática de Universidad adscrita al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutora

D. **Marcos Rodríguez Vega**, miembro del Grupo de Investigación en Criptología de la Universidad de La Laguna, como cotutor

C E R T I F I C A N

Que la presente memoria titulada:

“Análisis comparativo de modelos para la detección de audios deepfake”

ha sido realizada bajo su dirección por D. **Samuel Pérez López**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 15 de julio de 2026

Agradecimientos

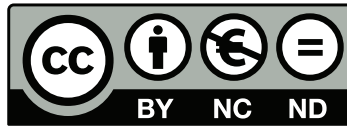
Este Trabajo de Fin de Grado no solo representa el esfuerzo del desarrollo e investigación presentado, sino también el final de una etapa, probablemente la más complicada y exigente de mi vida.

Este proyecto no habría sido posible sin mis tutores, Pino y Marcos, a quienes agradezco sinceramente por su tiempo, correcciones y apoyo durante todo el proceso y elaboración de este trabajo.

A mis amigos, por haber estado ahí, preocupándose genuinamente por mí en los momentos de mayor presión. Su ayuda para desconectar fue imprescindible para recuperar la motivación, recargar energías y poder seguir adelante.

A mis padres, por absolutamente todo. Gracias por su amor y apoyo incondicional, por estar siempre pendientes de mí y cuidarme en cada momento. Su esfuerzo diario para darme facilidades y tranquilidad sin pedir nada a cambio fue lo que me ha permitido llegar hasta aquí. Este logro es tan mío como suyo.

Licencia



© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional.

Resumen

El auge de los sistemas de síntesis y conversión de voz mediante redes neuronales profundas ha generado una importante amenaza para la veracidad de la comunicación oral. Esta investigación aborda la detección de audios deepfake mediante el diseño, entrenamiento y evaluación de un conjunto de clasificadores predictivos, tomando como referentes los benchmarks ASVspoof 2019 y 2021.

La metodología parte de la extracción de características acústicas en los dominios del tiempo y la frecuencia, sobre las que se entrenan modelos bajo distintos paradigmas de aprendizaje: tres clasificadores clásicos (Regresión Logística, SVM y XGBoost); una CNN de cinco bloques y las arquitecturas profundas ResNet y ResNeXt, integrando en todas ellas mecanismos de atención SE; una CRNN que registra la evolución temporal de la señal mediante capas LSTM (GRU bidireccional); y finalmente Wav2Vec, modelo transformer preentrenado mediante aprendizaje autosupervisado sobre la señal de onda cruda.

La evaluación emplea las métricas estándar de ASVspoof: EER y min t-DCF. En 2019 LA eval, los clasificadores clásicos establecen la base de referencia, destacando CQCC al superar a LFCC con un EER de 13,800 %. Las redes convolucionales superan notablemente esta base, confirmando la superioridad de los bloques SE frente a sus versiones estándar; la mejor CNN registró un EER de 11,400 % y un min t-DCF de 0,984. Wav2Vec ofrece la mejor respuesta global y la mayor resiliencia frente a la compresión de canal en ASVspoof 2021, con las mejores métricas del estudio: EER de 4,960 % y min t-DCF de 0,674.

Para una visualización más accesible que en terminal, se ha implementado una interfaz web en Streamlit que integra funciones para cargar audios, analizar espectrogramas y alternar entre modelos de inferencia, mostrando como diagnóstico final el veredicto de clasificación, su confianza y la latencia según la complejidad de la arquitectura.

Palabras clave: Deepfakes de audio, Síntesis de voz, ASVspoof, Aprendizaje profundo, Redes neuronales convolucionales, Aprendizaje autosupervisado, Detección de spoofing.

Abstract

The rise of voice synthesis and conversion systems developed through deep neural networks has generated a significant threat to the veracity of oral communication. This investigation addresses audio deepfake detection through the design, training, and evaluation of a set of predictive classifiers, taking the ASVspoof 2019 and 2021 benchmarks as a reference.

The methodology begins with the extraction of acoustic features in the time and frequency domains, on which models are trained under different learning paradigms: three classical classifiers (Logistic Regression, SVM, and XGBoost); a five-block CNN and the deep architectures ResNet and ResNeXt, all integrating SE attention mechanisms; a CRNN that captures the temporal evolution of the signal through LSTM layers (Bidirectional GRU); and finally Wav2Vec, a transformer model pretrained via self-supervised learning on the raw waveform signal.

Evaluation relies on the standard ASVspoof metrics: EER and min t-DCF. On the 2019 LA eval, classical classifiers establish the baseline, with CQCC standing out by outperforming LFCC with an EER of 13,800 %. Convolutional networks notably surpass this baseline, confirming the superiority of SE blocks over their standard counterparts; the best CNN recorded an EER of 11,400 % and a min t-DCF of 0,984. Wav2Vec delivers the best overall performance and the greatest resilience against channel compression in ASVspoof 2021, achieving the study's top metrics: an EER of 4,960 % and a min t-DCF of 0,674.

For a more accessible visualisation than a terminal, a web interface has been implemented in Streamlit that integrates functions to upload audio, analyse spectrograms, and switch between inference models, displaying as final diagnostic the classification verdict, its confidence, and the latency according to architectural complexity.

Keywords: Audio deepfakes, Voice synthesis, ASVspoof, Deep learning, Convolutional neural networks, Self-supervised learning, Spoofing detection.

Índice general

Índice de figuras	iii
Índice de tablas	iv
Índice de listados	v
Notación y acrónimos	vi
Capítulo 1 Introducción	1
1.1 Contexto y problemática	1
1.2 Motivación y objetivos del trabajo	1
Capítulo 2 Marco teórico y estado del arte	3
2.1 Estado del arte en detección de audio sintético	3
2.1.1 El desafío de evaluación ASVspoof	3
2.1.2 Evolución de los sistemas de detección	3
2.1.3 Desafíos actuales de robustez e integración de sistemas	4
2.2 El audio como señal digital	4
2.3 Aprendizaje automático	4
2.4 Aprendizaje profundo	5
Capítulo 3 Procesamiento de señales de audio	6
3.1 El audio como señal	6
3.2 Características del dominio temporal	7
3.3 La Transformada de Fourier	8
3.3.1 Intuición y transformada compleja	8
3.3.2 La Transformada Discreta de Fourier	10
3.3.3 La Transformada Rápida de Fourier	10
3.4 La Transformada de Fourier de Tiempo Corto	11
3.4.1 Segmentación y Ventaneo de la Señal	11
3.4.2 El spectral leakage y la ventana de Hann	11
3.4.3 Ecuación de la transformada y principio de incertidumbre de Gabor	12
3.5 La Transformada Wavelet	12
3.6 El espectrograma de Mel	13
3.7 Teoría del cepstrum y acústica del habla	14
3.8 Coeficientes Cepstrales en Frecuencia Mel, Lineal y Q Constante	15
Capítulo 4 Entorno de evaluación y benchmark ASVspoof	18
4.1 Corpus de datos y particiones oficiales	18
4.2 Métricas de evaluación	19
4.2.1 Tasas de error	19

4.2.2	Funciones de coste de detección	20
4.3	Protocolo de evaluación	22
Capítulo 5	Desarrollo de modelos y clasificación	23
5.1	Análisis de las representaciones de entrada	24
5.1.1	Artefactos espectrales en señales sintéticas	24
5.1.2	El espectrograma logarítmico y la normalización estándar	24
5.2	Modelos clásicos de <i>machine learning</i>	26
5.2.1	Regresión Logística	26
5.2.2	Máquinas de vectores de soporte con kernel RBF	27
5.2.3	XGBoost	27
5.3	Redes neuronales convolucionales	28
5.3.1	Fundamentos teóricos y principios de la convolución	28
5.3.2	Diseño y configuración de la red convolucional base	29
5.3.3	Bloques de atención Squeeze-and-Excitation	29
5.3.4	Redes residuales con atención por canal	30
5.4	Redes Neuronales Convolucionales Recurrentes	31
5.5	Aprendizaje autosupervisado y el modelo Wav2Vec 2.0	32
5.6	Entrenamiento y estrategia experimental	32
Capítulo 6	Configuración experimental y resultados	34
6.1	Entorno de ejecución, evaluación y despliegue	34
6.2	Análisis comparativo de resultados por corpus	35
6.2.1	Generalización consistente en modelos wav2vec 2.0	35
6.2.2	Rendimiento de arquitecturas convolucionales y clásicas	37
6.2.3	Variación del rendimiento entre conjuntos <i>dev</i> y <i>eval</i>	38
6.3	Eficiencia computacional	38
6.4	Robustez ante el desplazamiento de dominio	40
Capítulo 7	Conclusiones y líneas futuras	42
7.1	Conclusiones	42
7.2	Líneas futuras	43
Capítulo 8	Conclusions and Future Research	44
8.1	Conclusions	44
8.2	Future Research	45
Capítulo 9	Presupuesto	46
Apéndice		48
Bibliografía		49

Índice de figuras

3.1	Descomposición DWT (db4, 4 niveles) de una locución de 16 kHz.	13
3.2	Espectrogramas STFT-dB de muestras <i>bonafide</i> y sintéticas.	14
3.3	Banco de 80 filtros triangulares Mel.	14
3.4	Espectrograma de coeficientes cepstrales MFCC.	16
3.5	Espectrograma de coeficientes cepstrales LFCC	16
3.6	Espectrograma de coeficientes cepstrales CQCC	17
5.1	Visión del pipeline del sistema	23
5.2	Espectrogramas de entrada a la CNN normalizados mediante <i>z-score</i>	26
5.3	Arquitectura de la red CNN.	29
5.4	Bloque de conexión residual.	30
6.1	Evolución del EER por modelo a través de los corpus de evaluación.	41

Índice de tablas

6.1	Resultados en la partición de desarrollo (<i>Dev</i> · 2019 LA).	35
6.2	Resultados en la partición de evaluación (<i>Eval</i> · 2019 LA).	36
6.3	Resultados en la partición de evaluación (<i>Eval</i> · 2021 LA).	36
6.4	Resultados en la partición de evaluación (<i>Eval</i> · 2021 DF).	37
6.5	Eficiencia computacional de los sistemas.	39
6.6	Latencia total y rendimiento por sistema.	39
6.7	Evolución del EER (%) a través de los corpus de evaluación.	40
9.1	Presupuesto de equipos y licencias.	46
9.2	Coste de mano de obra.	47
9.3	Coste total del proyecto.	47

Índice de listados

4.1	Cálculo del EER (<code>src/metrics.py</code>).	21
4.2	Cálculo del minDCF (<code>src/metrics.py</code>).	21
5.1	Generación del espectrograma de entrada a la CNN (<code>src/features.py</code>).	25
5.2	Guardado de checkpoint ante pérdida de validación no finita (<code>src/pipeline.py</code>). .	33

Notación y acrónimos

Notación matemática

Símbolo	Significado
A	Amplitud de una onda sinusoidal (Ecuación 3.1)
f	Frecuencia, en hercios (Hz)
T	Periodo de una señal / periodo de muestreo
φ	Fase de una onda o de un coeficiente de Fourier
$s(k), x(n)$	Muestra k -ésima o n -ésima de la señal discreta
t	Índice temporal continuo, o umbral de decisión sobre la puntuación de spoof (Capítulo 4)
K	Tamaño del frame (en muestras)
$c = a + ib$	Número complejo con parte real a y parte imaginaria b
$ c , \gamma$	Módulo y argumento (fase) de un número complejo en notación polar
$\hat{g}(f), \hat{x}(k)$	Coeficiente de la Transformada de Fourier (continua/discreta) para la frecuencia f o el bin k
N	Número de muestras temporales / tamaño del bloque de la FFT-DFT
$w(k), w(n)$	Función de ventaneo
m	Índice de frame en la STFT
H	Hop size (desplazamiento entre frames consecutivos, en muestras)
$S(m, k)$	Coeficiente espectral complejo de la STFT en el frame m y el bin k
$Y(m, k)$	Espectrograma de potencia, $Y(m, k) = S(m, k) ^2$
τ, s (Wavelet)	Traslación temporal y escala en la Transformada Wavelet
ψ^*	Complejo conjugado de la wavelet madre
F, F^{-1}	Operador de la Transformada de Fourier y su inversa (usados en la definición del cepstrum)
$E(t), H(t)$	Pulso glotal (excitación) y respuesta en frecuencia del tracto vocal, respectivamente

Símbolo	Significado
$\text{score}(x)$	Puntuación de spoof asignada por el clasificador a la muestra x
t^*	Umbral óptimo que minimiza $ FAR(t) - FRR(t) $ (punto de EER)
$C_{\text{miss}}, C_{\text{fa}}$	Coste de un falso rechazo y coste de una falsa aceptación (DCF / t-DCF)
P_{target}	Probabilidad a priori de que un audio sea bonafide
$\pi_{\text{tar}}, \pi_{\text{non}}, \pi_{\text{spoof}}$	Probabilidades a priori de locutor objetivo, impostor humano y ataque de síntesis (t-DCF)
w, b	Vector de pesos y sesgo de un modelo lineal / regresión logística
$\sigma(\cdot)$	Función sigmoide (también usada para la desviación estándar en BatchNorm, según contexto)
γ (RBF)	Parámetro de anchura del kernel RBF en la SVM
λ	Coeficiente de regularización L2 (XGBoost, cobertura de hoja)
p_i	Probabilidad predicha por el modelo para la muestra i (XGBoost)
μ, σ (z-score)	Media y desviación estándar de la matriz espectral, usadas en la normalización z-score
ε	Constante pequeña que evita la división por cero (z-score, BatchNorm)
$a^{(l)}, W^{(l)}, b^{(l)}$	Activación, pesos y sesgo de la capa l de una red neuronal
L	Función de pérdida (entropía cruzada binaria)
$*$	Operador de convolución
h_t, c_t	Estado oculto y estado de celda de una RNN/LSTM en el paso temporal t
f_t, i_t, o_t	Puertas de olvido, entrada y salida de una celda LSTM
$\odot$	Producto elemento a elemento (Hadamard)
z_c	Descriptor escalar del canal c tras el <i>squeeze</i> en un bloque SE
$u_c, \tilde{u}_c$	Mapa de características del canal c antes y después de reescalarlo (bloque SE)
$F(x), H(x)$	Función residual aprendida y transformación deseada en una conexión residual (ResNet)
T (temperatura)	Factor de calibración de temperatura aplicado a los logits de Wav2Vec 2.0 antes del softmax

Lista de acrónimos

Sigla	Significado
AASIST	Audio Anti-Spoofing using Integrated Spectro-Temporal graph attention networks; sistema de detección basado en atención espectro-temporal sobre grafos
ADC	Analog-to-Digital Converter (Conversión Analógico-Digital)
AE	Amplitude Envelope (Envolvente de Amplitud)
ASV	Automatic Speaker Verification (Verificación Automática de Locutor)
ASVspoof	Automatic Speaker Verification Spoofing and Countermeasures Challenge
BCE	Binary Cross-Entropy (Entropía Cruzada Binaria); función de pérdida usada con logits (BCEWithLogitsLoss)
BN	Batch Normalization (Normalización por Lotes)
CM	Countermeasure (Contramedida); sistema que detecta el ataque de spoofing
CNN	Convolutional Neural Network (Red Neuronal Convolutacional)
CQCC	Constant Q Cepstral Coefficients (Coeficientes Cepstrales en Q Constante)
CQT	Constant-Q Transform (Transformada Q Constante)
CRNN	Convolutional Recurrent Neural Network (Red Neuronal Convolutacional-Recurrente)
CUDA	Compute Unified Device Architecture; plataforma de cómputo paralelo de NVIDIA para GPU
DAC	Digital-to-Analog Converter (Conversión Digital-Analógica)
DCF	Detection Cost Function (Función de Coste de Detección)
DCT	Discrete Cosine Transform (Transformada Coseno Discreta)
DF	Deepfake in the Wild; partición de evaluación de ASVspoof 2021
DFT	Discrete Fourier Transform (Transformada Discreta de Fourier)
DL	Deep Learning (Aprendizaje Profundo)
DWT	Discrete Wavelet Transform (Transformada Wavelet Discreta)
EER	Equal Error Rate (Tasa de Igual Error)
FAR	False Acceptance Rate (Tasa de Falsa Aceptación)
FFT	Fast Fourier Transform (Transformada Rápida de Fourier)
FLAC	Free Lossless Audio Codec; formato de compresión de audio sin pérdida
FRR	False Rejection Rate (Tasa de Falso Rechazo)

Sigla	Significado
FT	Fourier Transform (Transformada de Fourier)
GAN	Generative Adversarial Network (Red Generativa Adversaria)
GMM	Gaussian Mixture Model (Modelo de Mezcla de Gaussianas)
GPU	Graphics Processing Unit (Unidad de Procesamiento Gráfico)
GRU	Gated Recurrent Unit (Unidad Recurrente Compuertada)
HuBERT	Hidden-unit BERT; modelo de representación del habla preentrenado mediante SSL
IA	Inteligencia Artificial
IDFT	Inverse Discrete Fourier Transform (Transformada Discreta de Fourier Inversa)
LA	Logical Access (Acceso Lógico); partición/escenario de ataque de ASVspoof
LFCC	Linear Frequency Cepstral Coefficients (Coeficientes Cepstrales en Frecuencia Lineal)
LR	Logistic Regression (Regresión Logística)
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients (Coeficientes Cepstrales en Frecuencia Mel)
minDCF	Minimum Detection Cost Function (Coste de Detección Mínimo)
ML	Machine Learning (Aprendizaje Automático)
MLP	Multilayer Perceptron (Perceptrón Multicapa)
PCM	Pulse Code Modulation (Modulación por Impulsos Codificados)
RBF	Radial Basis Function (Función de Base Radial); kernel usado en la SVM
ReLU	Rectified Linear Unit (Unidad Lineal Rectificada)
ResNet	Residual Network (Red Residual)
ResNeXt	Extensión de ResNet con convoluciones agrupadas (cardinalidad)
RMS	Root Mean Square (Energía Cuadrática Media)
RNN	Recurrent Neural Network (Red Neuronal Recurrente)
SE	Squeeze-and-Excitation; bloque de atención por canal
SGD	Stochastic Gradient Descent (Descenso de Gradiente Estocástico)
SSL	Self-Supervised Learning (Aprendizaje Autosupervisado)
STFT	Short-Time Fourier Transform (Transformada de Fourier de Tiempo Corto)

Sigla	Significado
SVM	Support Vector Machine (Máquina de Vectores de Soporte)
t-DCF	Tandem Detection Cost Function (Función de Coste de Detección en Tándem)
TTS	Text-to-Speech (Síntesis de voz)
VC	Voice Conversion (Conversión de voz)
XGBoost	eXtreme Gradient Boosting (Potenciación del Gradiente Extremo)
ZCR	Zero Crossing Rate (Tasa de Cruce por Cero)

Modelos, sistemas y corpus citados

Nombre	Descripción
Wav2Vec 2.0	Modelo transformer preentrenado mediante SSL sobre la forma de onda cruda; adaptado (fine-tuned) para la detección bonafide/spoof
ResNet-SE / ResNeXt-SE	Arquitecturas residuales que integran bloques de atención SE
VITS	Sistema neuronal de síntesis de voz citado como ejemplo de generación de audio de alta calidad
HiFi-GAN	Vocoder neuronal basado en GAN citado como ejemplo de síntesis de voz de alta calidad
RawNet2	Sistema de detección de spoofing que procesa directamente la forma de onda cruda
VALL-E	Modelo de clonación de voz <i>zero-shot</i> citado en las líneas futuras
WavLM	Modelo fundacional de habla preentrenado mediante SSL, propuesto como línea futura de comparación
HuggingFace Transformers	Librería empleada para cargar y ajustar el modelo Wav2Vec 2.0
Streamlit	Framework de Python usado para desarrollar el frontend/demo interactiva del proyecto
librosa	Librería de Python empleada para el procesamiento de señales de audio
scikit-learn	Librería de Python empleada para los modelos de machine learning
PyTorch	Framework de deep learning empleado para entrenar las arquitecturas CNN, CRNN y Wav2Vec 2.0

Capítulo 1

Introducción

1.1. Contexto y problemática

Históricamente, reconocer si una voz era auténtica era evidente, era suficiente simplemente con escucharla. El acento, el ritmo, el tono o las imperfecciones naturales del habla humana actuaban como un identificador auditivo difícil de falsificar con medios convencionales. Sin embargo, esa certeza se rompió con la consolidación de las redes generativas.

Los recientes avances en modelos de síntesis de voz basados en redes neuronales, en particular los *vocoders* neuronales, las arquitecturas GAN y los modelos de difusión, han llevado la calidad del audio sintético a un nivel donde la diferenciación a partir de la percepción humana ya no puede considerarse un mecanismo de defensa fiable. Los sistemas como VITS, HiFi-GAN o modelos de clonación de voz con repositorios públicos demuestran que producir un audio con la voz de cualquier persona requiere, a día de hoy, minutos de cómputo y unos pocos segundos de audio real de referencia.

Esta capacidad generativa tiene consecuencias directas sobre la confianza entre personas y entre personas e instituciones. Un mensaje de voz de un familiar pidiendo dinero urgente o una instrucción de un ejecutivo ordenando una transferencia son vectores de ataque que explotan la confianza que culturalmente depositamos en la voz y que no tenemos en cuenta. El término *deepfake* agrupa este conjunto de contenidos falsificados con IA que son indistinguibles de la señal original para un oyente promedio.

Mientras que los deepfakes de vídeo e imagen han concentrado mayormente la atención mediática, los de audio presentan una superficie de ataque especialmente peligrosa por tres razones: su producción es computacionalmente más barata, su distribución puede realizarse mediante canales de voz convencionales como llamadas telefónicas o mensajes de voz, y los sistemas ASV que protegen servicios bancarios o de control de acceso son vulnerables a ellos por diseño si no incorporan una contramedida adicional.

1.2. Motivación y objetivos del trabajo

Los sistemas ASV modernos alcanzan tasas de error bajísimas frente a impostores humanos, pero su rendimiento cae drásticamente con los ataques de síntesis o conversión de voz. Para hacer frente a esta amenaza, surge el desafío *ASVspoof*, una competición que, desde su primera edición en 2015, proporciona bancos de datos estandarizados y métricas comunes: la EER y la t-DCF.

Las ediciones de 2019 y 2021 del desafío cubren tres categorías de ataque: síntesis de voz o TTS, VC y reproducción (*replay*). En 2021, el banco de datos incorpora además condiciones de canal degradado

que se acercan más al escenario real, donde el audio deepfake llega comprimido o transmitido a través de telefonía. Este trabajo usa ambas ediciones como escenario de evaluación.

El objetivo principal de este proyecto es diseñar, implementar y evaluar un sistema de detección de deepfakes de audio que cubra el proceso completo, desde la señal cruda hasta la predicción, comparando diferentes aproximaciones basadas en las métricas de rendimiento estandarizadas. En concreto, se plantean los siguientes objetivos específicos:

1. Estudiar el procesamiento de señales de audio para extraer representaciones compactas y discriminativas de la forma de onda (*waveform*), en particular STFT, STFT con normalización *Z-score*, MFCC, LFCC y CQCC.
2. Implementar y entrenar modelos de clasificación con distinta complejidad:
 - Modelos clásicos de *machine learning* (Regresión Logística, SVM y XGBoost) aplicados sobre vectores de características.
 - Arquitecturas de aprendizaje profundo basadas en convoluciones: CNN de 5 bloques, ResNet-SE y ResNeXt-SE.
 - Un CRNN que combina bloques CNN con una GRU bidireccional para capturar tanto las anomalías espectrales como la evolución temporal y secuencial del habla.
 - Un modelo basado en transformers (*Wav2Vec 2.0*), preentrenado mediante aprendizaje autosupervisado sobre la señal cruda.
3. Evaluar el rendimiento de todos los sistemas sobre ASVspoof 2019 y 2021 con las métricas estándar del campo (EER, min-tDCF), analizando también la latencia de predicción y el tiempo de extracción de características para evaluar su eficiencia en escenarios reales.
4. Desarrollar una interfaz interactiva en Streamlit que permita a un usuario cargar un audio y obtener en tiempo real la predicción del modelo y una visualización de las características extraídas.

Capítulo 2

Marco teórico y estado del arte

A continuación, se introduce el estado del arte en detección de *spoofing* de voz para contextualizar el proyecto, junto con una presentación conceptual y breve de las técnicas de procesamiento de señal y de los modelos de aprendizaje automático y profundo empleados. Este capítulo funciona como marco de referencia: el desarrollo matemático y de implementación de cada técnica se reserva para el Capítulo 3, Capítulo 4 y Capítulo 5.

2.1. Estado del arte en detección de audio sintético

2.1.1. El desafío de evaluación ASVspoof

La detección de audio sintético empezó con la primera edición del desafío *ASVspoof* en 2015 [1]. Antes de esa fecha, los sistemas ASV se evaluaban frente a impostores humanos, sin contemplar el audio generado por máquina como vector de ataque.

El desafío ha ido evolucionando en sus sucesivas ediciones, aumentando la complejidad de los ataques y las condiciones de evaluación. La edición de 2019 [2] es especialmente relevante porque unificó los tres tipos de ataque: TTS, VC y *replay*. La del 2021 [3] incorporó además condiciones de canal degradado (compresión de códec y transmisión telefónica) que se acercan al escenario de amenaza real, donde el audio *deepfake* raramente llega sin sufrir alteraciones en el canal.

2.1.2. Evolución de los sistemas de detección

Los primeros sistemas del desafío combinaron representaciones de señal clásicas con modelos probabilísticos. El *baseline* oficial de ASVspoof utiliza coeficientes LFCC clasificados con una GMM [4], y resultó competitivo frente a varios tipos de ataque, mostrando que las frecuencias altas del espectro son particularmente discriminativas para detectar artefactos.

Con la expansión del aprendizaje profundo, los sistemas evolucionaron hacia representaciones aprendidas directamente de los datos. La aplicación de redes neuronales convolucionales sobre espectrogramas de Mel [5] demostró que las arquitecturas diseñadas para visión artificial podían trasladarse al dominio del audio. Los sistemas más recientes como RawNet2 [6] procesan directamente la forma de onda cruda, y AASIST [7] destaca por procesar la señal combinando un codificador con un modelo de atención espectro-temporal basado en grafos, consiguiendo los mejores resultados en ASVspoof 2021. Por otro lado, la adaptación de modelos preentrenados mediante aprendizaje autosupervisado, en

particular Wav2Vec 2.0 [8], ha demostrado una generalización mucho mayor ante ataques no vistos, que es el escenario más crítico en la práctica.

2.1.3. Desafíos actuales de robustez e integración de sistemas

La capacidad de generalización de un sistema de detección ante condiciones no vistas en entrenamiento es a día de hoy la línea de trabajo más activa del campo. Se mostró [9] que los detectores entrenados sobre ASVspoof pierden gran parte de su rendimiento al evaluarse sobre grabaciones *in the wild* recogidas de fuentes reales (redes sociales, podcasts), demostrando que el propio diseño de ASVspoof, con ataques generados en condiciones controladas de laboratorio, no captura toda las variaciones del escenario real. Esta observación motiva la comparación entre los corpus 2019 LA y 2021 DF que se desarrolla en el Capítulo 6.

Para reducir el sobreajuste a los sistemas de ataque vistos en entrenamiento, técnicas de aumento de datos como RawBoost [10] introducen variaciones sintéticas de canal, ruido y respuestas al impulso directamente sobre la forma de onda, mejorando la generalización de modelos *end-to-end* como RawNet2 y AASIST sin necesidad de datos de ataque adicionales.

Por otro lado, el desafío SASV [11] plantea evaluar conjuntamente la ASV y la CM como un único sistema, en lugar de tratarlos como módulos independientes conectados en tándem, que es el esquema que asume la definición de la t-DCF, asumiendo que optimizar cada componente por separado no garantiza el rendimiento óptimo del sistema. Finalmente, otras iniciativas como el desafío ADD [12] amplían el problema más allá de las condiciones de ASVspoof, lo que confirma que este es un campo en pleno desarrollo con muchos desafíos pendientes.

2.2. El audio como señal digital

El sonido se origina en las vibraciones de un objeto, que propagan una onda mecánica con tres tipos de información perceptible: la frecuencia, la intensidad y el timbre. Para que un ordenador pueda procesarlo, esa onda continua debe convertirse en datos digitales mediante un proceso de ADC.

Una vez digitalizado, el audio admite dos perspectivas de análisis complementarias. Las características del dominio temporal como la AE, la RMS o la ZCR describen cómo varía la amplitud de la señal segundo a segundo, pero no permiten identificar qué frecuencias la componen. Las características del dominio frecuencial sí lo hacen, y son más informativas para la detección de *deepfakes*. El timbre, los formantes y los artefactos de síntesis quedan distribuidos en el espectro y no localizados en el tiempo. La herramienta central para pasar de un dominio al otro es la FT, cuyo desarrollo matemático detallado, junto con el resto de representaciones espectrales empleadas en este trabajo (STFT, Wavelet, MFCC, LFCC, CQCC) se explica en el Capítulo 3.

2.3. Aprendizaje automático

El aprendizaje automático es la rama de la IA centrada en identificar patrones en los datos y generalizar ese conocimiento a observaciones nuevas. Puede ser supervisado, no supervisado o por refuerzo. Este trabajo emplea aprendizaje supervisado, ya que cada fragmento de audio llega etiquetado como

bonafide o *spoof*. El reto central de cualquier modelo supervisado es el compromiso sesgo-varianza, ya que un modelo demasiado simple no captura la relación real de los datos (alto sesgo), mientras que uno demasiado complejo memoriza el conjunto de entrenamiento en lugar de generalizar (alta varianza, *overfitting*). La validación cruzada (*cross-validation*) es la herramienta estándar para estimar la capacidad de generalización y disminuir el sobreajuste sin malgastar datos.

Se evalúan tres clasificadores clásicos sobre los vectores de características extraídos de la señal. La regresión logística adapta la regresión lineal a la clasificación binaria mediante una función sigmoide, aportando un modelo simple e interpretable. Las SVM buscan, en cambio, el hiperplano que maximiza el margen entre clases, recurriendo a funciones *kernel* cuando los datos no son linealmente separables. Por último, XGBoost construye, de forma secuencial, un conjunto de árboles de decisión donde cada árbol corrige los residuos del anterior (*gradient boosting*), añadiendo regularización L1/L2 y procesamiento en paralelo de forma nativa. El desarrollo matemático y los hiperparámetros concretos de los tres modelos se presentan en el Capítulo 5.

2.4. Aprendizaje profundo

El aprendizaje profundo es un subcampo del aprendizaje automático basado en redes neuronales artificiales organizadas en capas, donde cada neurona combina linealmente sus entradas y aplica una función de activación no lineal (a día de hoy casi siempre ReLU) para poder aprender relaciones complejas. El entrenamiento ajusta los pesos mediante retropropagación (*backpropagation*) y SGD, minimizando una función de pérdida de entropía cruzada, que es adecuada para la clasificación binaria que se aborda aquí.

Por tanto, este proyecto evalúa varias familias de arquitecturas. Las CNN introducen la convolución como operación central, ideal para detectar patrones espaciales en espectrogramas ya que sus filtros permiten reconocer características visuales sin importar en qué parte aparezcan. Las arquitecturas ResNet [13] añaden conexiones residuales que permiten entrenar redes mucho más profundas sin degradación del rendimiento, mientras que los bloques Squeeze-and-Excitation [14] reponderan cada canal según su relevancia. La ResNeXt [15] por otro lado combina ambas ideas con convoluciones agrupadas. Las CRNN incorporan capas LSTM [16] (GRU bidireccional) para capturar la evolución temporal de la señal, algo que una CNN por sí sola no modela bien. Por último, los modelos de tipo Transformer [17], y en particular Wav2Vec 2.0 [8], sustituyen la recurrencia por mecanismos de *self-attention* y se preentrenan mediante SSL sobre la forma de onda cruda, lo que les da una capacidad de generalización superior ante ataques no vistos. El desarrollo completo de estas arquitecturas, incluyendo las ecuaciones de cada bloque, la configuración de entrenamiento y los hiperparámetros, se presentan en el Capítulo 5.

Capítulo 3

Procesamiento de señales de audio

Detectar un deepfake de audio empieza en la señal. Los algoritmos generativos cometen errores matemáticos que dejan marcas en el espectro, sobre todo en las frecuencias altas. Si el procesamiento inicial de la señal difumina esa información antes de que llegue al clasificador, los artefactos que delatan al deepfake se perderán de manera irreversible. Por eso, el tratamiento de señal es la primera decisión de diseño con consecuencias directas sobre el rendimiento final del sistema.

3.1. El audio como señal

El sonido se produce cuando un objeto vibra y hace que las moléculas del aire oscilen, cambiando la presión y creando una onda mecánica que se propaga por el espacio. Una onda mecánica es una oscilación que viaja donde transfiere energía de un punto a otro sin que el medio se desplace de forma permanente.

Las ondas sonoras pueden clasificarse en dos grandes grupos [18]: las periódicas, con un patrón que se repite en el tiempo pudiendo ser simples (una única senoide) o complejas (superposición de varias), y las aperiódicas, que carecen de una periodicidad definida. Una onda de sonido lleva tres tipos de información: la frecuencia (el tono), la intensidad (el volumen) y el timbre (la calidad que distingue dos fuentes sonoras). La fórmula matemática de una onda sinusoidal pura es:

$$y(t) = A \sin(2\pi ft + \varphi) \quad (3.1)$$

donde A es la amplitud, f la frecuencia en hercios, t el tiempo y φ la fase inicial. La frecuencia y el periodo T están relacionados inversamente:

$$f = \frac{1}{T} \quad (3.2)$$

El desfase entre dos ondas de la misma frecuencia se denomina fase y determina si se combinan o se anulan entre sí. El pitch y la frecuencia son logarítmicos y se representan habitualmente en octavas: duplicar la frecuencia equivale a subir una octava.

El sonido es una señal analógica, por lo que sus valores de tiempo y amplitud son continuos. Para que un ordenador pueda procesar el audio se tiene que transformar esa onda continua en datos digitales mediante la ADC. Mientras que en la señal analógica existen infinitos valores posibles para el eje temporal y el de amplitud, la señal digital se compone de una secuencia de valores discretos, por lo

que el sistema solo puede representar un número finito de ellos. El resultado final de este proceso se denomina PCM. La conversión consta de dos procesos:

El muestreo (*sampling*) registra el valor de amplitud de la señal a intervalos de tiempo regulares a una tasa constante (*sampling rate*). Cuanto mayor sea la tasa de muestreo, más precisa será la captura. El aliasing (solapamiento) y los artefactos ocurren cuando la frecuencia de la señal de entrada supera la frecuencia de Nyquist, que es exactamente la mitad del *sampling rate*. Para evitarlo se aplican filtros *anti-aliasing* antes del ADC.

La cuantización (*quantization*) asigna a cada muestra el valor predefinido más cercano según la resolución del sistema, que está determinada por el número de bits (*bit depth*). A mayor resolución, menor error de cuantización. También existe el rango dinámico, que es la diferencia entre la mayor y menor señal que el sistema puede representar. Cuanto mayor es el *bit depth*, mayor es el rango dinámico.

La cadena completa de una grabación es: la onda mecánica llega al micrófono, que oscila y genera una señal eléctrica; esa señal pasa a la tarjeta de sonido, que aplica los filtros *anti-aliasing* y ejecuta el ADC, produciendo la señal PCM. El DAC convierte la señal digital en una señal eléctrica que estimula las membranas del altavoz, generando la onda mecánica [18]. En el *machine learning* el flujo suele ser extraer las características en el dominio temporal o frecuencial y pasarlas a un algoritmo clásico, mientras que en el aprendizaje profundo se alimenta el espectrograma directamente a una red neuronal que aprende sus propias representaciones.

3.2. Características del dominio temporal

Una característica del dominio temporal mide cómo cambia la amplitud de la señal cada segundo [18]. Para poder tratar los datos, la señal se divide en ventanas cortas llamadas frames, y sobre cada frame se calculan estadísticas. Hay algunas características del dominio del tiempo que se aplican a cada frame:

La AE mide, dentro de cada frame, cuál es la muestra con mayor amplitud:

$$AE_t = \max_{k=t \cdot K}^{(t+1) \cdot K - 1} s(k) \quad (3.3)$$

Donde t es el índice del frame, K el tamaño del frame y $s(k)$ el valor de la muestra k -ésima. Proporciona una idea del volumen de la señal, es muy sensible a los valores atípicos y aporta la detección de inicio de eventos sonoros (*onset detection*).

La RMS también es un indicador de volumen de la señal pero es menos sensible a valores atípicos que la AE, ya que promedia la energía de todas las muestras del frame. Se usa para identificar nuevos segmentos en una señal de audio y para la clasificación de géneros musicales:

$$RMS_t = \sqrt{\frac{1}{K} \cdot \sum_{k=t \cdot K}^{(t+1) \cdot K - 1} s(k)^2} \quad (3.4)$$

La ZCR es el número de veces que una señal cruza el eje horizontal por frame:

$$ZCR_t = \frac{1}{2} \cdot \sum_{k=t \cdot K}^{(t+1) \cdot K - 1} |sgn(s(k)) - sgn(s(k+1))| \quad (3.5)$$

Cuando la amplitud $s(k)$ es positiva devuelve $+1$; si es negativa, -1 ; si es exactamente 0, devuelve 0. Para cada par de muestras consecutivas k y $k+1$, si los valores tienen el mismo signo el resultado es 0 (no ha cruzado). Cuando son de signos opuestos el valor absoluto es 2 y la suma acumula ese cruce. Esto permite reconocer los sonidos percusivos de los sonidos con tono, y también se usa para la estimación de tono monofónico y la decisión voz/no voz.

Estas características temporales tienen una limitación fundamental, ya que no permiten identificar qué frecuencias componen la señal. Para detectar deepfakes, los artefactos de síntesis más reveladores no están en la variación temporal de la amplitud, sino en el dominio frecuencial.

3.3. La Transformada de Fourier

3.3.1. Intuición y transformada compleja

La intuición de la Transformada de Fourier es descomponer el sonido complejo en sus componentes de la frecuencia [18]. El objetivo es pasar del dominio del tiempo al dominio de la frecuencia. Lo primero que se hace es comparar la señal original con diferentes sinusoides de distintas frecuencias. La senoide de análisis es:

$$\sin(2\pi \cdot (ft - \varphi)) \quad (3.6)$$

Se obtiene una magnitud y una fase. La magnitud nos dice cómo de similares son la señal original y el senoide, es decir, cuanto mayor la magnitud, mayor la similaridad. Cuando la frecuencia coincide con la de la señal original y la fase está bien cuadrada, multiplicando la señal original por la sinusoidal las áreas generadas son mayormente positivas y su integral es grande. Si la fase no es la correcta, las áreas serían negativas aunque la frecuencia sea la misma, por lo que su similaridad sería nula. Cuanto más se acerca la fase y la frecuencia, más área positiva hay para indicar cuánto se parecen las señales. La fórmula para la fase óptima dada una frecuencia es:

$$\varphi_f = \operatorname{argmax}_{\varphi \in [0,1)} \left(\int s(t) \cdot \sin(2\pi \cdot (ft - \varphi)) \cdot dt \right) \quad (3.7)$$

y la magnitud se calcula como:

$$d_f = \max_{\varphi \in [0,1)} \left(\int s(t) \cdot \sin(2\pi \cdot (ft - \varphi)) \cdot dt \right) \quad (3.8)$$

El proceso total de la Transformada de Fourier consiste en repetir este proceso para cada frecuencia, obteniendo así el espectro completo de la señal.

Teniendo en cuenta que la FT tiene dos parámetros, la forma compacta es codificar la magnitud (número real) y la fase en un único número complejo. Un número complejo se expresa como $c = a + ib$, con $a, b \in \mathbb{R}$, donde a es la parte real y ib la parte imaginaria. Con coordenadas polares:

$$\gamma = \arctan\left(\frac{b}{a}\right), \quad |c| = \sqrt{a^2 + b^2} \quad (3.9)$$

$$a = |c| \cdot \cos(\gamma), \quad b = |c| \cdot \sin(\gamma) \quad (3.10)$$

$$c = |c| \cdot (\cos(\gamma) + i \sin(\gamma)) \quad (3.11)$$

Aplicando la fórmula de Euler, $e^{i\gamma} = \cos(\gamma) + i \sin(\gamma)$, se puede trazar en sentido antihorario la unidad circular y reescribir las coordenadas polares de forma compacta:

$$c = |c| \cdot e^{i\gamma} \quad (3.12)$$

donde $|c|$ escala la distancia desde el origen y $e^{i\gamma}$ fija la dirección del número en el plano complejo.

El coeficiente de la Transformada de Fourier en números complejos es:

$$c_f = \frac{d_f}{\sqrt{2}} \cdot e^{-i2\pi\varphi_f} \quad (3.13)$$

Hay un coeficiente para cada frecuencia que descomponemos de la señal original. La transformada compleja mapea de reales a complejos: $\hat{g} : \mathbb{R} \rightarrow \mathbb{C}$, con $\hat{g}(f) = c_f$. La definición de la Transformada de Fourier compleja es:

$$\hat{g}(f) = \int g(t) \cdot e^{-i2\pi ft} dt \quad (3.14)$$

Desarrollando $e^{i\gamma} = \cos(\gamma) + i \sin(\gamma)$, el coeficiente se puede descomponer en su parte real e imaginaria:

$$\hat{g}(f) = \int g(t) \cdot \cos(-2\pi ft) dt + i \int g(t) \cdot \sin(-2\pi ft) dt \quad (3.15)$$

De aquí se obtienen directamente la magnitud y la fase:

$$d_f = \sqrt{2} \cdot |\hat{g}(f)|, \quad \varphi_f = -\frac{\gamma_f}{2\pi} \quad (3.16)$$

El proceso contrario es la IDFT, que permite pasar del dominio de la frecuencia al dominio del tiempo, reconstruyendo la señal sobreponiendo los sinusoides pesados por su magnitud y fase relativas, parecido a lo que hace un sintetizador.

3.3.2. La Transformada Discreta de Fourier

El audio analógico es una señal continua. Para que un ordenador pueda buscar los patrones anómalos de un deepfake se tiene que transformar en datos digitales estructurados. La forma continua de la transformada se convierte a su versión discreta:

$$\hat{x}(f) = \int g(t) \cdot e^{-i2\pi ft} dt \longrightarrow \hat{x}(f) = \sum_n x(n) \cdot e^{-i2\pi fn} \quad (3.17)$$

Para conseguir esta transformación el sistema asume que la señal tiene una duración finita en un intervalo. El audio continuo se divide en ventanas fijas y, para que la transformación sea invertible y matemáticamente eficiente, el número de componentes frecuenciales a evaluar M se iguala al número de muestras temporales N :

$$k = [0, M - 1] = [0, N - 1] \quad (3.18)$$

La ecuación definitiva de la DFT realiza un sumatorio discreto desde la muestra 0 hasta la $N - 1$:

$$\hat{x}(k/N) = \sum_{n=0}^{N-1} x(n) \cdot e^{-i2\pi n \frac{k}{N}} \quad (3.19)$$

Visualmente la DFT hace una aproximación por rectángulos para calcular el área bajo la curva de la señal, transformando el dominio del tiempo en contenedores de frecuencia. El resultado devuelve un vector indexado por k , que se mapea a una magnitud física expresada en hercios usando la relación:

$$F(k) = \frac{k}{NT} = \frac{k \cdot S_T}{N} \quad (3.20)$$

donde T es el periodo de muestreo y S_T es el *sampling rate*. Al hacer la gráfica del espectro de magnitudes se crea una redundancia simétrica. Esto significa que el espectro genera un efecto espejo exacto a partir de su punto medio, correspondiente a la Frecuencia de Nyquist:

$$k = \frac{N}{2} \Rightarrow F\left(\frac{N}{2}\right) = \frac{s_r}{2} \quad (3.21)$$

Ya que todo lo que se calcula por encima de este límite es un reflejo exacto que no aporta más información, se descarta la mitad superior del vector, optimizando el almacenamiento.

3.3.3. La Transformada Rápida de Fourier

Calcular la DFT tiene una complejidad algorítmica de $\mathcal{O}(N^2)$. Mediante esta metodología se generaría un cuello de botella que colapsaría los tiempos de cómputo, por lo que la librería de desarrollo utiliza la FFT, que aprovecha las simetrías de las ondas para reducir el coste a $\mathcal{O}(N \log_2 N)$, con la condición de que el tamaño del bloque sea una potencia exacta de dos.

3.4. La Transformada de Fourier de Tiempo Corto

3.4.1. Segmentación y Ventaneo de la Señal

El propósito de la STFT es aplicar una FFT de forma local por segmentos (*frames*) [18]. Primero se segmenta la señal tomando un fragmento del audio y se le aplica el *windowing*:

$$x_w(k) = x(k) \cdot w(k) \quad (3.22)$$

El window size es la cantidad de *samples* a la que se aplica el *windowing*, mientras que el frame size es el número de *samples* que se consideran en cada *chunk* de la señal cuando se segmenta y se le pasa a la STFT. Normalmente el window size y el frame size coinciden, aunque pueden no hacerlo en casos específicos. El hop size es cuántos *samples* se deslizan a la derecha cuando hay un nuevo frame.

3.4.2. El spectral leakage y la ventana de Hann

Al extraer un bloque de la señal y aplicarle la FFT se asume implícitamente que ese bloque es periódico. Si no contiene un número entero de periodos, los extremos presentan una discontinuidad artificial llamada spectral leakage (derrame espectral) que genera componentes de alta frecuencia inexistentes en la señal original.

Para minimizarlo, se aplica una función de windowing a cada frame, que multiplica las muestras por una envolvente con un máximo en el centro y que decrece a cero en los extremos. La más famosa y usada antes de aplicar la transformada de Fourier es la ventana de Hann:

$$w(k) = 0,5 \cdot \left(1 - \cos\left(\frac{2\pi k}{K-1}\right) \right), \quad k = 1 \dots K \quad (3.23)$$

que da como resultado la señal con windowing aplicado:

$$s_w(k) = s(k) \cdot w(k), \quad k = 1 \dots K \quad (3.24)$$

Con esto surge otro problema, y es que al anular las muestras de los extremos se pierde señal. Para solventarlo se superponen frames. Como el hop size es menor que el frame size, los extremos de una ventana quedan en el centro de la siguiente y la información no se pierde.

3.4.3. Ecuación de la transformada y principio de incertidumbre de Gabor

La comparación entre DFT y STFT muestra la diferencia estructural:

$$\text{DFT: } \hat{x}(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-i2\pi n \frac{k}{N}} \quad (3.25)$$

$$\text{STFT: } S(m, k) = \sum_{n=0}^{N-1} x(n + mH) \cdot w(n) \cdot e^{-i2\pi n \frac{k}{N}} \quad (3.26)$$

En la ecuación de la STFT m representa el número de frame. La cantidad mH (frame actual $\times$ hop size) es el *sample* que empieza en el frame actual, es decir, la muestra de inicio de cada ventana. Después se multiplica por la función de windowing $w(n)$ y por último se multiplica por el tono puro $e^{-i2\pi nk/N}$. Mientras que la DFT genera un *vector* espectral con N coeficientes de Fourier complejos, la STFT genera una *matriz* espectral con coeficientes de Fourier complejos indexados por (m, k) .

Para ver cómo cambia el espectro a lo largo del tiempo, la STFT corta el audio en ventanas fijas y calcula la FFT en cada una. Aquí ocurre el Principio de Incertidumbre de Gabor. El problema es que no es posible tener una resolución perfecta en tiempo y en frecuencia a la vez con una ventana fija. Si se usa una ventana muy pequeña, se sabe exactamente cuándo ocurre algo, pero no hay suficientes muestras para saber qué frecuencia exacta es. Si se usa una ventana muy grande, se pueden calcular los hercios exactos, pero al promediar un fragmento tan largo se pierde el momento en el que ocurrió un cambio rápido.

Los algoritmos generativos de IA suelen introducir micro-cortes o saltos de fase raros que duran milisegundos; pero, a la vez, para analizar el tono de voz artificial se necesita observar bien los graves y la STFT no puede dar ambas cosas a la vez.

3.5. La Transformada Wavelet

La DWT rompe el límite del principio de Gabor, sustituyendo las ondas seno y coseno infinitas de Fourier por una Wavelet Madre [19], que es una función matemática pequeña y localizada en el tiempo:

$$F(\tau, s) = \frac{1}{\sqrt{|s|}} \int_{-\infty}^{+\infty} f(t) \cdot \psi^*\left(\frac{t - \tau}{s}\right) dt \quad (3.27)$$

donde τ es la traslación temporal, s es la escala y ψ^* el complejo conjugado de la wavelet madre. En vez de usar una ventana de tamaño fijo, se estira o encoge y desplaza esa onda a lo largo del audio. Para las frecuencias altas, la wavelet se contrae y permite analizar el audio con una precisión temporal elevada. Para las frecuencias bajas, se dilata y captura ciclos completos con una alta precisión frecuencial para evaluar el tono.

Como en un ordenador no se pueden aplicar estiramientos infinitos, se usa la DWT, que restringe las escalas en potencias de 2 e implementa el análisis con un banco de filtros recursivos:

- Filtro paso-bajo: elimina los agudos y se queda con la estructura gruesa. A la salida se obtienen los *Coeficientes de Aproximación* (A).
- Filtro paso-alto: elimina los graves y se queda con los detalles rápidos y los cortes bruscos. A la salida se obtienen los *Coeficientes de Detalle* (D).

Al filtrar a la mitad el teorema de Nyquist permite hacer un *downsampling* (diezmado) por 2, descartando una de cada dos muestras para optimizar el rendimiento sin perder información. En la Figura 3.1 se observa de manera visual este proceso para una locución de 16 kHz, mostrando la distribución de los coeficientes de aproximación (A_4) y de detalle (D_1-D_4) resultantes.

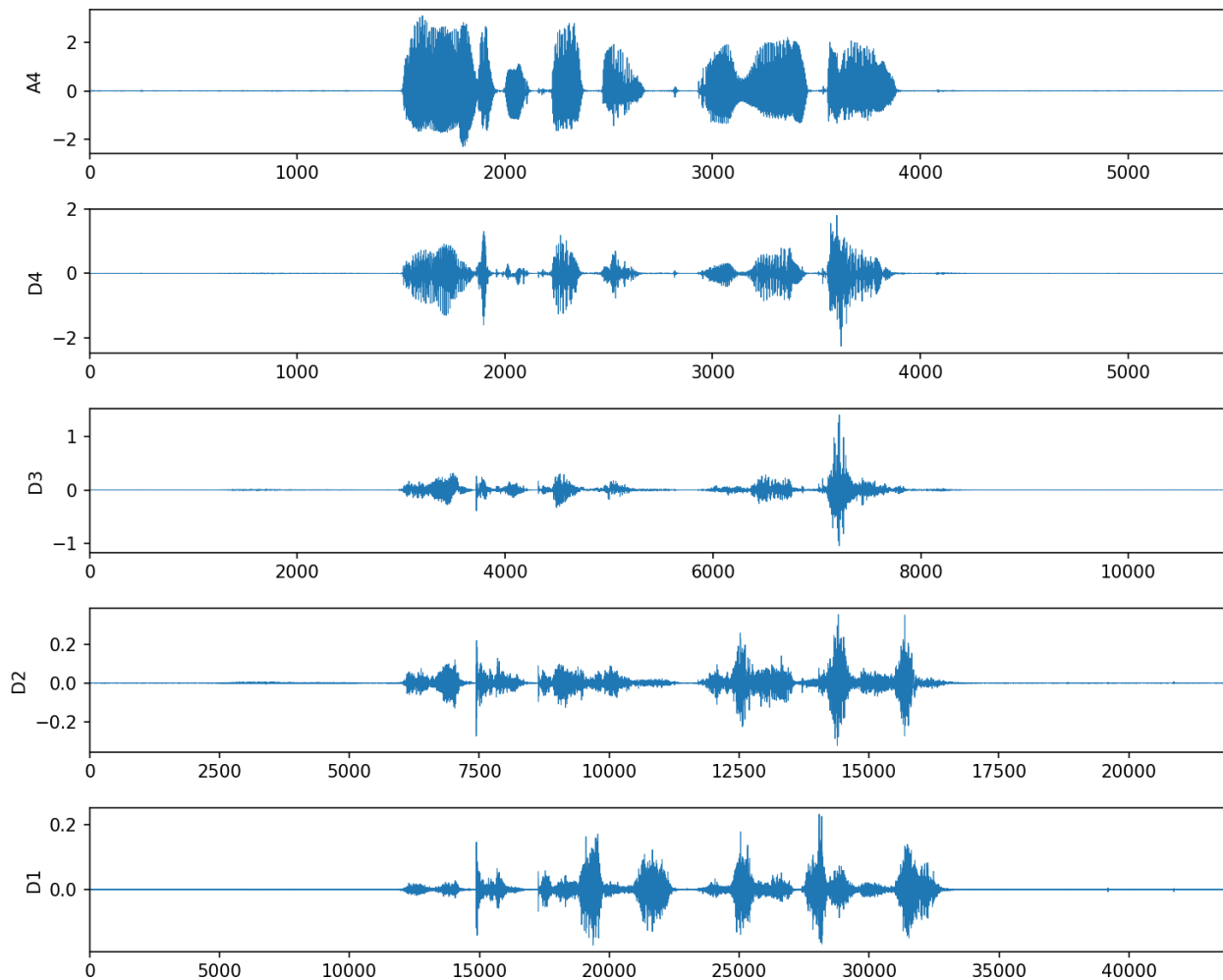


Figura 3.1: Descomposición DWT (db4, 4 niveles) de una locución de 16 kHz.

3.6. El espectrograma de Mel

Para visualizar el sonido a través de un espectrograma se usa un mapa de calor con la siguiente fórmula para obtener el espectrograma de potencia:

$$Y(m, k) = |S(m, k)|^2 \quad (3.28)$$

Con los espectrogramas lineales como el del STFT, existe el problema de que los humanos perciben la frecuencia de forma logarítmica [18]. En la Figura 3.2 se presenta una comparación de estos espectrogramas, en la cual se aprecia que el audio *bonafide* (Figura 3.2a) presenta una continuidad armónica natural, la muestra sintética (Figura 3.2b) muestra imperfecciones del algoritmo de generación, como patrones repetitivos en las altas frecuencias (> 4 kHz) y discontinuidades horizontales.

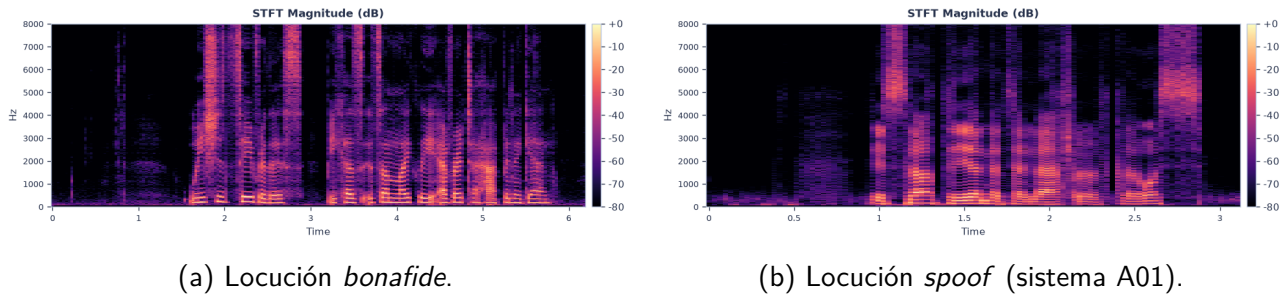


Figura 3.2: Espectrogramas STFT-dB de muestras *bonafide* y sintéticas.

La escala de Mel se basa en el timbre y también es logarítmica. Para representar el tiempo y la frecuencia con una representación relevante para los humanos, se usa el espectrograma de Mel. Primero se extrae el STFT, se convierte la amplitud a dBs y se transforman las frecuencias a la escala de Mel, donde se elige un número de bandas, se construyen filtros y se aplican al espectrograma. El banco de filtros contiene filtros triangulares solapados, donde las frecuencias centrales están distribuidas uniformemente, situándose más juntas en las frecuencias bajas y más separadas en las altas. Esto se puede apreciar en la Figura 3.3 para un banco de 80 filtros triangulares diseñado para audio de 16 kHz ($n_{\text{fft}} = 1024$), donde se observa una mayor densidad y concentración en el rango de bajas frecuencias para imitar la resolución del oído.

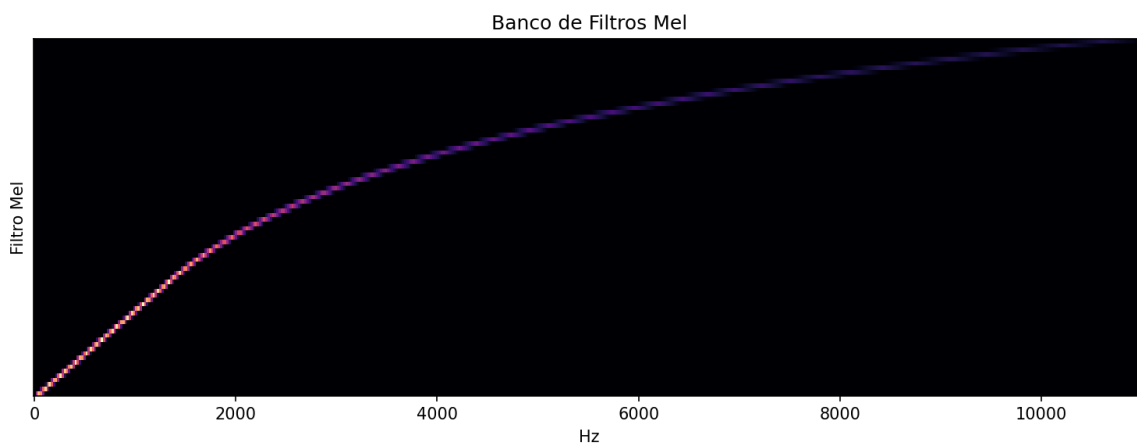


Figura 3.3: Banco de 80 filtros triangulares Mel.

3.7. Teoría del cepstrum y acústica del habla

Existen varios términos que se han inventado para adaptarlos al caso: *Cepstrum* / *Quefrequency* / *Liftering* / *Rhamonic*. Estas palabras son inversiones parciales de: *Spectrum* / *Frequency* / *Filtering*

/ *Harmonic* [18]. El cepstrum es básicamente el espectro de un espectro, o la Transformada de Fourier inversa de una Transformada de Fourier. Se puede computar así:

$$C(x(t)) = F^{-1}[\log(F[x(t)])] \quad (3.29)$$

donde F denota la Transformada de Fourier y F^{-1} su inversa. La variable del dominio resultante se llama *quefrequency*.

Se puede entender el habla y descomponerla de la siguiente manera. Para definirla, la voz humana es la convolución de la respuesta en frecuencia del tracto vocal con el pulso glotal:

$$X(t) = E(t) \cdot H(t) \quad (3.30)$$

donde $X(t)$ es la señal de voz, $E(t)$ el pulso glotal (excitación de las cuerdas vocales) y $H(t)$ la respuesta en frecuencia del tracto vocal. Aplicando logaritmos, la convolución se convierte en suma, y la separación generada permite aislar el tracto vocal del pulso glotal:

$$\log(X(t)) = \log(E(t) \cdot H(t)) \quad (3.31)$$

$$\log(X(t)) = \log(E(t)) + \log(H(t)) \quad (3.32)$$

Por último se hace el liftering, que consiste en aplicar la IDFT y filtrar el pulso glotal para quedarnos solo con la respuesta en frecuencia del tracto vocal, que es lo que interesa para el reconocimiento de voz.

3.8. Coeficientes Cepstrales en Frecuencia Mel, Lineal y Q Constante

Una vez explicada la base teórica de cómo se separan matemáticamente el pulso glotal y el tracto vocal, el siguiente paso es trasladar este modelo a un algoritmo práctico. Las MFCC son la herramienta clásica más utilizada para conseguirlo, ya que combinan la separación matemática del espectro con la forma real en la que el oído humano percibe el sonido. Específicamente, el proceso completo para calcular los MFCC se ejecuta mediante el siguiente flujo:

$$\text{waveform} \xrightarrow{\text{DFT}} \text{Log-Amplitude Spectrum} \xrightarrow{\text{Mel-Scaling}} \text{Mel bands} \xrightarrow{\text{DCT}} \text{MFCC} \quad (3.33)$$

Se utiliza la DCT en vez de la IDFT porque, aparte de ser una versión simplificada de la FT, se obtiene un coeficiente de valor real y se descorrelaciona la energía en diferentes bandas Mel reduciendo las dimensiones para representar el espectro, lo cual ayuda al proceso del *machine learning*.

Las ventajas de los MFCC residen en su capacidad para describir las estructuras grandes del espectro ignorando las finas, lo que les permite funcionar eficazmente en el procesamiento de voz y de música. Sus desventajas incluyen su vulnerabilidad ante entornos ruidosos, el elevado esfuerzo para su modelado y una baja eficiencia cuando se utilizan para tareas de síntesis. Los humanos somos

muy sensibles a pequeños cambios en los tonos graves, pero a partir de los 1000 Hz nos da igual si algo suena a 14000 Hz o a 15000 Hz porque nuestro oído los agrupa. Por eso, el banco de filtros Mel comprime y filtra los detalles de las frecuencias altas. Como ilustración de este resultado, en la Figura 3.4, se presenta un mapa de calor con los 20 coeficientes MFCC obtenidos de una muestra de ASVspoof 2019 LA, ordenados por orden cepstral en función del tiempo utilizando *frames* de 512 muestras.

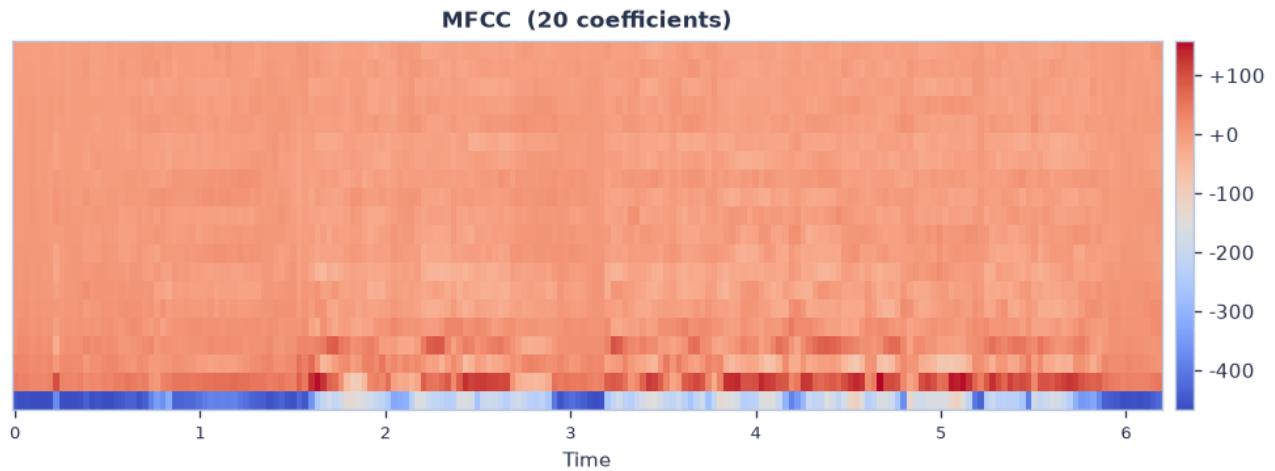


Figura 3.4: Espectrograma de coeficientes cepstrales MFCC.

Los LFCC son coeficientes cepstrales calculados a partir de la señal de audio, diseñados para capturar la información de todo el espectro de frecuencias por igual [18]. El proceso es prácticamente idéntico al de los MFCC, pero la diferencia radical está en el banco de filtros. Mientras que los MFCC aplican filtros de forma logarítmica, los LFCC aplican un banco de filtros estrictamente lineal a lo largo de todo el espectro de frecuencias. Este diseño se ilustra en la Figura 3.5, donde su resolución constante en el rango de alta frecuencia (> 4 kHz) resulta fundamental para preservar los artefactos espectrales generados por los *vocoders* neuronales, permitiendo identificar anomalías que la escala Mel pasaría por alto.

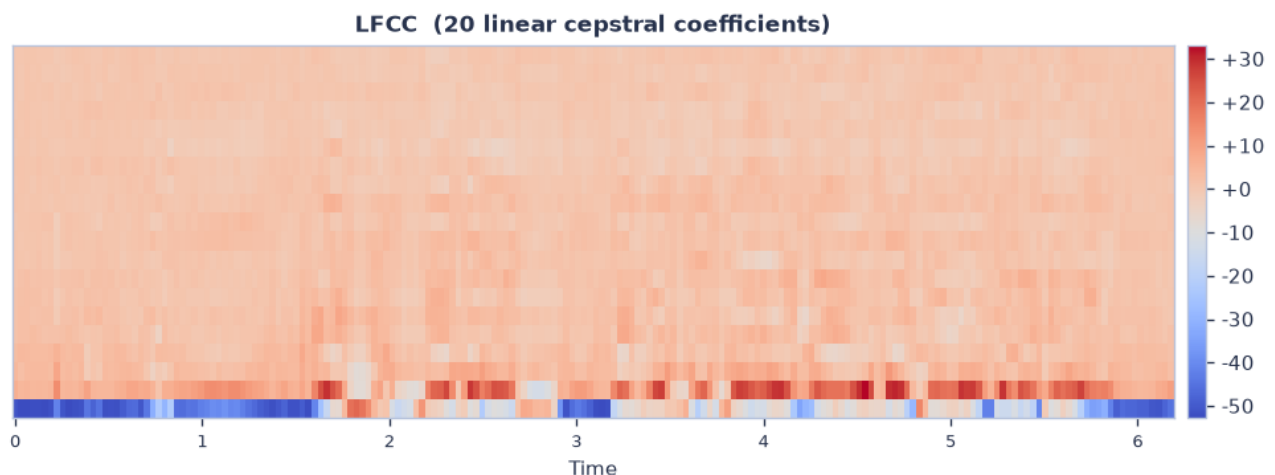


Figura 3.5: Espectrograma de coeficientes cepstrales LFCC

Al dibujar el audio digitalmente, la IA comete micro-errores como artefactos de cuantización, patrones repetitivos o ruidos sintéticos en las frecuencias más altas. Si se usa MFCC, el banco de filtros Mel

comprime y promedia esa zona destruyendo las estructuras espectrales finas y borrando las pistas del *deepfake*. Si se usa LFCC, los filtros mantienen la misma resolución reflejando anomalías artificiales. Estos son el estándar porque preservan la firma espectral que dejan los algoritmos generativos [20]. Al pasarle un vector LFCC a un clasificador clásico, se le dan datos con una resolución limpia de alta frecuencia para que pueda trazar un hiperplano que separe los audios reales de los sintéticos.

A diferencia de MFCC y LFCC, que procesan el audio dividiéndolo en bloques de tiempo fijos mediante la STFT y luego agrupan las frecuencias con bancos de filtros, el CQCC utiliza la Transformada Q Constante (CQT) [18].

La CQT adapta el tamaño de sus ventanas de forma dinámica. Usa ventanas de tiempo largas para analizar las frecuencias bajas con gran nitidez y ventanas de tiempo muy cortas para las frecuencias altas. Esto le da una resolución temporal perfecta para capturar los micro-ruídos, transiciones rápidas y pequeñas imperfecciones de fase.

Como la CQT ya distribuye las frecuencias de forma logarítmica, para obtener los coeficientes cepstrales basta con re-muestrear ese espectro a una escala uniforme, calcular el logaritmo de la energía y aplicar la DCT para separar sus componentes independientes. El resultado es un conjunto de coeficientes puros que describen la física exacta del sonido, permitiendo identificar anomalías que para nuestros oídos pasan completamente desapercibidas. En la Figura 3.6 se detalla el espectrograma resultante de estos coeficientes cepstrales extraídos mediante la CQT, donde se puede apreciar cómo el uso de ventanas de tamaño variable captura con máxima precisión esos cambios rápidos y las sutiles anomalías de fase que delatan el origen sintético de la señal.

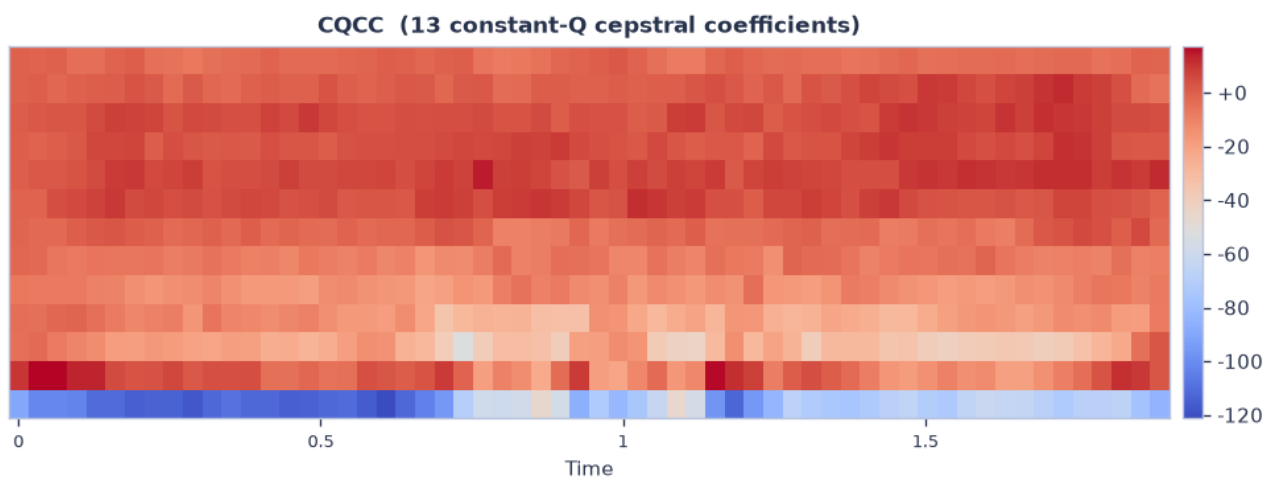


Figura 3.6: Espectrograma de coeficientes cepstrales CQCC

Capítulo 4

Entorno de evaluación y benchmark ASVspoof

Para que los resultados de un sistema de detección de deepfakes sean comparables con los de otros equipos y no dependan de qué audios se elijan para la prueba, hace falta un banco de datos público con etiquetas verificadas y una métrica de evaluación estándar. El desafío ASVspoof proporciona ese marco de evaluación que la comunidad investigadora ha adoptado como referencia desde 2015.

4.1. Corpus de datos y particiones oficiales

ASVspoof es una competición que nació con el objetivo de impulsar el desarrollo de CM capaces de proteger los sistemas de verificación frente a ataques de suplantación [2].

La edición de 2019 incorporó por primera vez ataques basados en redes neuronales profundas como vocoders, GAN y modelos de conversión de voz. Los ataques que evalúa el desafío se agrupan en tres categorías: síntesis de voz o TTS, VC y reproducción (*replay*). En la tarea de LA, el atacante puede inyectar el audio sintetizado en el sistema ASV sin limitaciones de canal, por lo que los ataques TTS y VC son los más relevantes.

La edición de 2021 amplió el escenario añadiendo dos nuevas particiones: *LA 2021*, que aplica condiciones de canal telefónico reales (codecs y ruido de transmisión) sobre los mismos tipos de ataque, y *DF 2021*, que recoge audios falsos en condiciones no controladas con una mayor variedad de ataques [3].

El corpus de 2019 en LA contiene audio a 16 kHz, 16 bits PCM, distribuido en formato FLAC (compresión sin pérdida que preserva los artefactos de cuantización del vocoder), y se divide en tres particiones:

- Entrenamiento (*train*): contiene audios de los sistemas de ataque A01–A06 y los audios *bonafide*. Todos los pesos de los modelos se aprenden exclusivamente sobre esta partición.
- Desarrollo (*dev*): son los mismos sistemas de ataque que *train*. Se usa para monitorizar la pérdida durante el entrenamiento de los modelos.
- Evaluación (*eval*): contiene ataques A07–A19, todos desconocidos durante el entrenamiento. Es la partición sobre la que se evalúan los resultados finales del Capítulo 6.

El protocolo oficial se distribuye como un fichero de texto con cinco columnas separadas por espacios:

SPEAKER_ID	AUDIO_FILE	-	ATTACK_SYSTEM	KEY
LA_0079	LA_T_1138215	-	-	bonafide
LA_0079	LA_T_1271820	-	A01	spoof

La columna KEY puede tener dos valores: *bonafide* (audio humano real) o *spoof* (audio sintético). La columna ATTACK_SYSTEM indica el sistema generativo para cada audio falso. Esta información se usa en el proyecto para construir una partición de validación con ataques no vistos (Apartado 4.3).

Las particiones de 2021 se usan únicamente para evaluación. Su función es medir la degradación que experimentan los sistemas entrenados en 2019 cuando se enfrentan a condiciones reales. El 2021 LA somete los mismos tipos de ataque TTS y VC a las transformaciones de canal que introduce la telefonía: codificación de voz (*codecs*), ruido de transmisión y efectos de la red. Un sistema que funcione bien en el de 2019 puede colapsar aquí si ha aprendido a confiar en artefactos espectrales que el codec borra. Por otro lado, 2021 DF recoge deepfakes capturados en condiciones no controladas con una variedad de sistemas de ataque mayor. Esto representa el escenario más cercano al real. La diferencia de rendimiento entre 2019 y 2021 es el resultado del experimento sobre el desplazamiento de dominio (*domain shift*).

4.2. Métricas de evaluación

4.2.1. Tasas de error

Los detectores de deepfakes producen una puntuación de *spoof* que es un número real donde los valores altos indican que el sistema cree que el audio es falso. Para convertir esa puntuación en una decisión binaria, se fija un umbral t . Si la puntuación supera t , el audio se declara *spoof*, si no lo supera, se declara *bonafide*. Desplazar t cambia el equilibrio entre dos tipos de error [21]:

La FRR es la proporción de audios *bonafide* que el sistema clasifica incorrectamente como *spoof*. Cuanto más alto se pone el umbral, menos *bonafide* se rechazan, por lo que *FRR* decrece con t :

$$FRR(t) = \frac{|\{x \in \text{bonafide} \mid \text{score}(x) \geq t\}|}{|\text{bonafide}|} \quad (4.1)$$

La FAR es la proporción de audios *spoof* que el sistema deja pasar como *bonafide*. Al subir el umbral, más *spoof* no alcanzan t y se cuelan, por lo que *FAR* crece con t . FRR decrece mientras FAR crece, de forma que ambas curvas se cruzan exactamente en un punto:

$$FAR(t) = \frac{|\{x \in \text{spoof} \mid \text{score}(x) < t\}|}{|\text{spoof}|} \quad (4.2)$$

El EER es la tasa de error en el punto donde $FAR = FRR$. Como las curvas son discretas, se calcula como la media aritmética de FAR y FRR en el umbral t^* que minimiza su diferencia:

$$EER = \frac{FAR(t^*) + FRR(t^*)}{2}, \quad t^* = \operatorname{argmin}_t |FAR(t) - FRR(t)| \quad (4.3)$$

La interpretación del EER es básica, si el sistema tiene un EER del 5 %, eso significa que en el peor umbral posible comete un 5 % de errores. Por ejemplo un detector que acierta al azar tendría $EER = 50 \%$. En este proyecto, el cálculo se realiza en un único paso después de ordenar las puntuaciones, contabilizando cuántos *bonafide* y cuántos *spoof* quedan por debajo del umbral en cada iteración.

4.2.2. Funciones de coste de detección

El EER evalúa el sistema en el umbral que lo favorece más, por lo que no es una métrica fiable del todo. La DCF incorpora los costes asimétricos de los dos tipos de error y las probabilidades de las clases. Su expresión normalizada es:

$$C_{\det}(t) = C_{\text{miss}} \cdot P_{\text{target}} \cdot FRR(t) + C_{\text{fa}} \cdot (1 - P_{\text{target}}) \cdot FAR(t) \quad (4.4)$$

donde C_{miss} es el coste de rechazar un audio real, C_{fa} es el coste de dejar pasar un *deepfake* y P_{target} es la probabilidad de que un audio sea *bonafide*. El minDCF es el mínimo de $C_{\det}$ después de optimizar el umbral de decisión entre todas las opciones posibles. El resultado se normaliza por el coste de la decisión más simple (aceptar o rechazar siempre):

$$\text{minDCF} = \frac{\min_t C_{\det}(t)}{C_{\text{default}}}, \quad C_{\text{default}} = \min(C_{\text{miss}} \cdot P_{\text{target}}, C_{\text{fa}} \cdot (1 - P_{\text{target}})) \quad (4.5)$$

Un $\text{minDCF} \leq 1$ indica que el detector mejora la decisión trivial. Los parámetros establecidos por el plan de evaluación de ASVspoof 2019 son:

$$P_{\text{target}} = 0,05 \quad C_{\text{miss}} = 1 \quad C_{\text{fa}} = 10 \quad (4.6)$$

El coste de aceptar un *deepfake* ($C_{\text{fa}} = 10$) es diez veces mayor que el de rechazar un audio real ($C_{\text{miss}} = 1$), lo que refleja que en aplicaciones de seguridad biométrica un falso positivo es mucho más peligroso que un falso negativo.

En un escenario real, el CM no trabaja solo. Su salida se encadena con la del sistema ASV que verifica la identidad del locutor. El audio tiene que superar primero el filtro de la CM y luego el del ASV para ser aceptado. En este sistema tándem hay tres tipos posibles de usuario con sus probabilidades: $\pi_{\text{tar}} + \pi_{\text{non}} + \pi_{\text{spoof}} = 1$, donde π_{tar} es la probabilidad de un locutor objetivo legítimo, π_{non} la de un impostor humano y π_{spoof} la de un ataque de síntesis. Esto genera cuatro escenarios de error que son la aceptación de un *deepfake* por ambos sistemas, el rechazo de un audio legítimo por la CM, o que el ASV rechace a un usuario objetivo o acepte a un impostor.

La t-DCF agrega el coste de todos estos errores ponderados por sus probabilidades y por los costes asignados a cada tipo de fallo [21]. En este análisis se consideran dos escenarios extremos para el modelo de CM:

- Un sistema CM que rechaza todo (*REJECT-ALL*): Nunca permite el acceso a ningún *deepfake*, pero a la vez bloquea la totalidad de los usuarios legítimos, con un coste asociado expresado como $t\text{-DCF}_{\text{REJECT-ALL}} = C_{\text{miss}}^{\text{cm}} \cdot \pi_{\text{tar}}$.

- Un sistema CM perfecto (*oracle*): Identifica con total precisión cada muestra, delegando el filtrado final completamente en el sistema ASV, cuyo coste se calcula mediante $t\text{-DCF}_{\text{IDEAL-CM}}(t) = C_{\text{miss}}^{\text{asv}} \cdot \pi_{\text{tar}} \cdot P_{\text{miss}}^{\text{asv}}(t) + C_{\text{fa}}^{\text{asv}} \cdot \pi_{\text{non}} \cdot P_{\text{fa}}^{\text{asv}}(t)$.

En la práctica, los datos públicos de ASVspoof no incluyen las puntuaciones del ASV, lo que hace imposible calcular la t-DCF completa. Por eso la competición usa el minDCF simplificado, donde las puntuaciones del ASV se fijan en las condiciones ($C_{\text{fa}}^{\text{asv}} = C_{\text{miss}}^{\text{cm}} = 1$, $\pi_{\text{spoof}} = 0,001$) y la fórmula se colapsa a la DCF normal.

Para garantizar la trazabilidad de los resultados del Capítulo 6, el EER y el minDCF se han implementado en `src/metrics.py` sin depender de librerías externas. El Listado 4.1 muestra la implementación del EER a partir de la definición de la Ecuación 4.3.

```

1 def calculate_eer(scores, labels):
2     total_bonafide = sum(1 for e in labels if e == 0)
3     total_spoof    = sum(1 for e in labels if e == 1)
4
5     pairs = sorted(zip(scores, labels), key=lambda p: p[0])
6     best_diff, best_eer = float("inf"), 1.0
7     bonafide_below = spoof_below = idx = 0
8     n = len(pairs)
9
10    while idx <= n:
11        frr = (total_bonafide - bonafide_below) / total_bonafide
12        far = spoof_below / total_spoof
13        diff = abs(far - frr)
14        if diff < best_diff:
15            best_diff, best_eer = diff, (far + frr) / 2.0
16        if idx == n:
17            break
18
19        current_score = pairs[idx][0]
20        while idx < n and pairs[idx][0] == current_score:
21            bonafide_below += (pairs[idx][1] == 0)
22            spoof_below    += (pairs[idx][1] == 1)
23            idx += 1
24
25    return best_eer

```

Listado 4.1: Cálculo del EER (`src/metrics.py`).

El minDCF se implementa en el Listado 4.2 a partir de la Ecuación 4.5, que reutiliza el mismo barrido de umbrales sustituyendo la métrica minimizada por el coste ponderado $C_{\text{det}}(t)$:

```

1 def calculate_min_dcf(scores, labels, p_target=0.05, c_miss=1.0, c_fa=10.0):
2     total_bonafide = sum(1 for e in labels if e == 0)
3     total_spoof    = sum(1 for e in labels if e == 1)
4     c_default = min(c_miss * p_target, c_fa * (1.0 - p_target))
5
6     pairs = sorted(zip(scores, labels), key=lambda p: p[0])
7     best_dcf = float("inf")
8     bonafide_below = spoof_below = idx = 0
9     n = len(pairs)
10
11    while idx <= n:

```

```

12     frr = (total_bonafide - bonafide_below) / total_bonafide
13     far = spoof_below / total_spoof
14     dcf = c_miss * p_target * frr + c_fa * (1.0 - p_target) * far
15     best_dcf = min(best_dcf, dcf)
16     if idx == n:
17         break
18
19     current_score = pairs[idx][0]
20     while idx < n and pairs[idx][0] == current_score:
21         bonafide_below += (pairs[idx][1] == 0)
22         spoof_below    += (pairs[idx][1] == 1)
23         idx += 1
24
25     return best_dcf / c_default

```

Listado 4.2: Cálculo del minDCF (src/metrics.py).

Ambas funciones comparten el mismo barrido de umbrales en $O(n \log n)$, lo que evita recalcular el orden de las puntuaciones cuando se necesitan las dos métricas para la misma fila de resultados.

4.3. Protocolo de evaluación

El diseño del experimento sigue tres principios que garantizan la validez de los resultados. En primer lugar, se mantiene una separación estricta de particiones. Los pesos de todos los modelos se ajustan únicamente sobre la partición *train* de 2019 LA, mientras que las particiones *dev*, *eval* de 2019 y las dos particiones de 2021 nunca se usan durante el entrenamiento ni para tomar decisiones de arquitectura.

En segundo lugar, la validación se apoya en ataques no vistos. El *dev* comparte los sistemas de ataque A01–A06 con *train*, lo que hace que los modelos CNN se saturen rápidamente sin haber aprendido nada útil sobre ataques nuevos. La parada temprana usa una partición de validación construida con los ataques A05 y A06 del propio entrenamiento, y la selección del checkpoint se hace sobre el error en esta partición de ataques no vistos, lo que da una señal de generalización mucho más fiable.

Por último, la evaluación se realiza en tres escenarios. Los resultados finales se reportan sobre 2019 LA eval, 2021 LA y 2021 DF, y el *domain shift* entre 2019 y 2021 es el criterio que permite determinar qué modelos son realmente útiles fuera del entorno controlado.

Capítulo 5

Desarrollo de modelos y clasificación

La arquitectura del sistema sigue una jerarquía de complejidad creciente, empezando por modelos clásicos de *machine learning* sobre vectores de características DSP, después redes convolucionales profundas sobre espectrogramas 2D, un modelo híbrido convolucional-recurrente y, por último, un *transformer* preentrenado de forma autosupervisada.

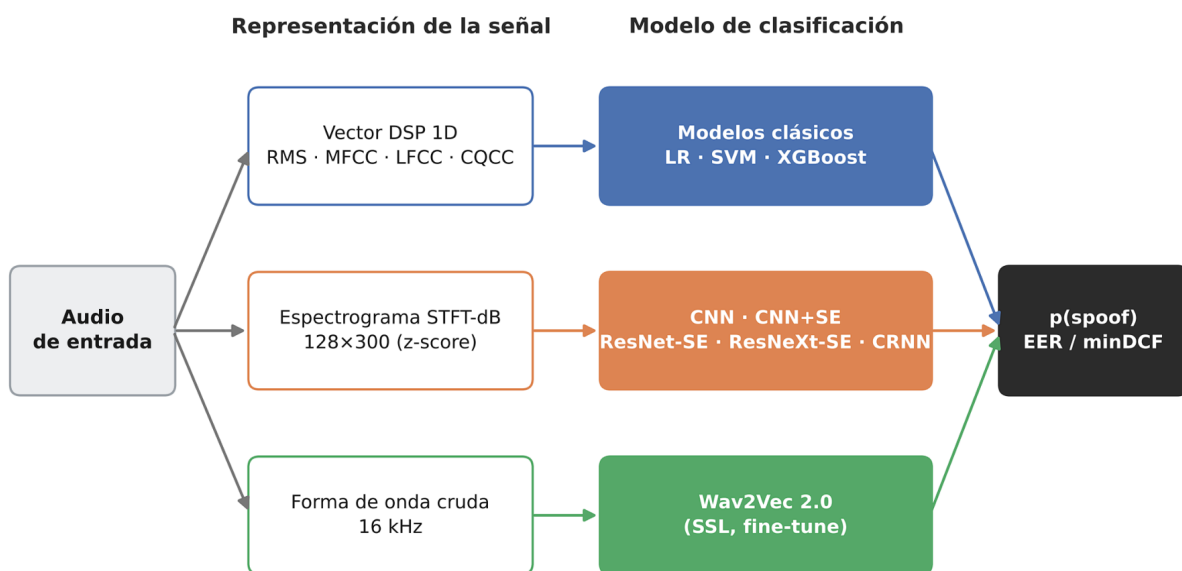


Figura 5.1: Visión del pipeline del sistema

Como se resume en la Figura 5.1, el mismo audio de entrada alimenta tres caminos de representación independientes:

- Vector 1D de características DSP: orientado a la alimentación de los modelos clásicos (Apartado 5.2).
- Espectrograma STFT-dB 2D: diseñado para la familia de arquitecturas convolucionales y recurrentes (Apartado 5.3-Apartado 5.4).
- Forma de onda cruda: procesada directamente por el modelo autosupervisado Wav2Vec 2.0 (Apartado 5.5).

Los tres enfoques convergen en una puntuación de *spoof* evaluada bajo las mismas métricas (EER y minDCF) sobre los mismos corpus de datos. Esto garantiza que la comparación analizada a lo largo del Capítulo 6 sea completamente homogénea y justa entre las distintas familias de modelos.

5.1. Análisis de las representaciones de entrada

5.1.1. Artefactos espectrales en señales sintéticas

La base de todo el sistema de detección parte de que los algoritmos generativos cometen errores en frecuencias que el oído humano no percibe pero que el espectro delata. La voz humana es el resultado de la convolución del pulso glotal con la respuesta en frecuencia del tracto vocal. Ese proceso produce espectrogramas con variaciones naturales, donde las bandas de resonancia cambian suavemente de un frame a otro, la energía decae de forma orgánica en las frecuencias altas y las transiciones fonéticas dejan marcas espectrales irregulares.

Un vocoder o modelo TTS genera el audio de manera diferente, ya que aprende una representación comprimida del espectro e intenta reconstruir la señal. Al hacerlo deja artefactos concretos [20]:

- Periodicidad artificial en alta frecuencia: los modelos neuronales suelen producir patrones repetitivos por encima de 4 kHz que no aparecen en la voz natural.
- Discontinuidad de fase entre frames: la síntesis frame a frame puede introducir micro-saltos que se ven como cortes horizontales bruscos en el plano tiempo-frecuencia.
- Supresión del ruido de excitación glotal: la voz tiene un componente de ruido en el pulso glotal, mientras que los sistemas TTS lo eliminan o modelan dejando las frecuencias altas demasiado limpias.
- Invariabilidad de timbre: los modelos de conversión de voz copian la identidad del locutor objetivo pero no siempre reproducen las micro-variaciones de amplitud, por lo que el espectrograma parece más plano que uno real.

Por estos factores, la representación espectral que se pase al clasificador debe mantener la información de alta frecuencia intacta. Por ello, los modelos clásicos usan LFCC en lugar de MFCC, y las CNN reciben el espectrograma STFT completo.

5.1.2. El espectrograma logarítmico y la normalización estándar

Para los modelos CNN, el audio de entrada se convierte en una representación bidimensional de tamaño fijo de 128×300 (bandas de frecuencia $\times$ frames temporales). El proceso completo de `get_spectrogram_matrix` en `src/features.py` es:

1. STFT con ventana de Hann: Se aplica la STFT con los siguientes parámetros: `n_fft=1024` y `hop_length=512` (50 % de solapamiento). El resultado es una matriz de magnitudes $513 \times T$ donde $513 = N_{\text{fft}}/2 + 1$, que son los contenedores de frecuencia hasta Nyquist.
2. Conversión a decibelios: La escala logarítmica comprime el rango dinámico de la magnitud lineal a un rango manejable para el descenso de gradiente. El *floor* de -80 dB sustituye el silencio completo ($-\infty$) por un valor finito:

$$S_{\text{dB}}(m, k) = 20 \cdot \log_{10} \left(\frac{|S(m, k)|}{\max_{m,k} |S(m, k)|} \right), \quad \text{limitado en } -80 \text{ dB} \quad (5.1)$$

3. Compresión del eje de frecuencia: El bin de Nyquist se descarta y los 512 restantes se agrupan de 4 en 4 promediando su energía, dando los 128 canales de frecuencia. Este conserva el ancho de banda completo [0, 8 kHz] por lo que la pérdida es de resolución, no de rango.
4. Ajuste del eje temporal: La señal se trunca con el valor del piso en decibelios hasta alcanzar exactamente 300 frames ($300 \times 512/16000 \approx 9,6s$). El padding con el piso dB equivale a añadir silencio sin introducir energía espuria.
5. Normalización mediante *z-score por audio*: Corrige las diferencias de volumen y condiciones de grabación del corpus. Si se pasan las magnitudes en bruto, el gradiente de la primera capa convolucional varía demasiado dependiendo del audio. La normalización elimina esa variación pasando los audios a la misma escala. Además, el *BatchNorm* recibe activaciones ya centradas en cero, lo que acelera su estabilización y permite tasas de aprendizaje mayores [22]. En la fórmula, μ y σ son la media y la desviación estándar de la matriz espectral, y ε previene la división por cero en señales de silencio:

$$\hat{S}(m, k) = \frac{S_{dB}(m, k) - \mu}{\sigma + \varepsilon} \quad (5.2)$$

El Listado 5.1 recoge la implementación de `get_spectrogram_matrix` en `src/features.py`, que traduce directamente los cinco pasos descritos en esta sección a código de producción.

```

1 def get_spectrogram_matrix(self, y):
2     magnitude = self._stft_magnitude(y) # (n_fft//2+1, T) = (513, T)
3
4     # --- Paso a decibelios (Ec. 5.1) ---
5     ref = float(magnitude.max()) or 1.0
6     matrix_db = librosa.amplitude_to_db(magnitude, ref=ref, top_db=80.0)
7
8     matrix_db = matrix_db[:-1, :]
9     n_total, n_frames = matrix_db.shape
10    factor = n_total // self.freq_bins
11    matrix_db = (
12        matrix_db[: factor * self.freq_bins, :]
13        .reshape(self.freq_bins, factor, n_frames)
14        .mean(axis=1)
15    )
16
17    floor_db = float(matrix_db.min())
18    if n_frames < self.time_frames:
19        pad = np.full((self.freq_bins, self.time_frames - n_frames), floor_db,
20                      dtype=matrix_db.dtype)
21        matrix_db = np.concatenate([matrix_db, pad], axis=1)
22    else:
23        matrix_db = matrix_db[:, : self.time_frames]
24
25    # --- Normalización z-score por audio (Ec. 5.2) ---
26    mean, std = matrix_db.mean(), matrix_db.std()
27    matrix_db = (matrix_db - mean) / (std + self._EPS)
28
29    return matrix_db.astype(np.float32)

```

Listado 5.1: Generación del espectrograma de entrada a la CNN (`src/features.py`).

En el código, la compresión del eje de frecuencia agrupa los 512 bins restantes en bloques de 4 mediante un promedio, en lugar de truncar el espectro. Esto reduce la resolución frecuencial pero conserva el ancho de banda completo de 0 a 8 kHz, incluida la zona de alta frecuencia donde los vocoders neuronales dejan sus artefactos. El relleno del eje temporal utiliza el propio suelo en decibelios de cada espectrograma (`floor_db`) en lugar de un cero, de forma que el *padding* equivale a añadir silencio real y no artefactos que distorsionaría el entrenamiento.

Este proceso genera las características de entrada de 128×300 para la CNN. Como se muestra en la Figura 5.2, la muestra *bonafide* (Figura 5.2a) mantiene una estructura limpia y orgánica, mientras que la *spoof* (Figura 5.2b) revela patrones de regularidad artificial en alta frecuencia inducidos por el *vocoder*.

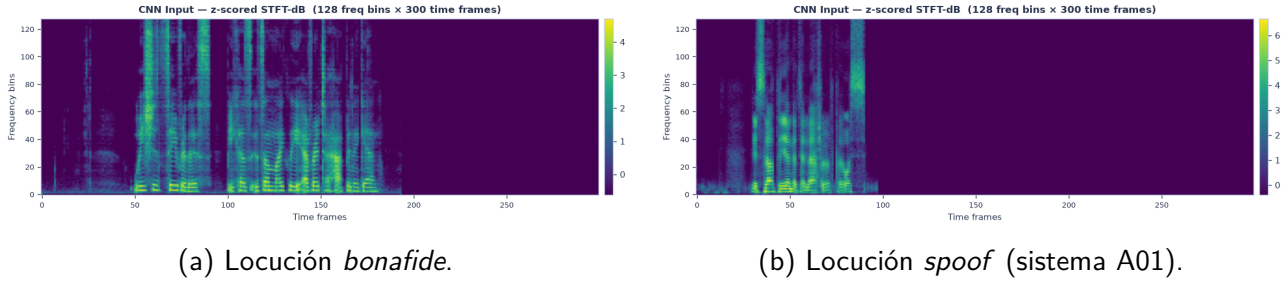


Figura 5.2: Espectrogramas de entrada a la CNN normalizados mediante *z-score*.

5.2. Modelos clásicos de *machine learning*

Los modelos clásicos operan sobre vectores 1D de características DSP. Cada audio se representa como la concatenación de la media y la varianza de sus coeficientes a lo largo del tiempo, es decir, un vector que codifica la “firma espectral media” del audio [23]. El proceso es el siguiente: audio $\rightarrow$ características DSP $\rightarrow$ vector 1D $\rightarrow$ clasificador $\rightarrow p(\text{spoof})$.

5.2.1. Regresión Logística

La regresión lineal predice valores continuos, de forma que con las características $\mathbf{x}$ estima un número real. La regresión logística adapta esa idea a la clasificación binaria modelando la probabilidad mediante la función sigmoide:

$$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (5.3)$$

La sigmoide convierte cualquier valor real en $[0, 1]$, que se interpreta directamente como $p(\text{spoof})$. El modelo se entrena con *log-loss* (entropía cruzada binaria) y regularización L2 con $C = 1,0$ que penaliza pesos grandes y previene el sobreajuste. Dado el desbalance del corpus, se activa `class_weight='balanced'` para que la pérdida pese inversamente la frecuencia de cada clase [23]. Un `StandardScaler` antes del clasificador asegura que todos los coeficientes cepstrales lleguen al optimizador `lbfgs` en la misma escala.

5.2.2. Máquinas de vectores de soporte con kernel RBF

En el *machine learning* un modelo con mucho sesgo (como la regresión lineal simple) no se ajusta bien a la relación real, y un modelo con mucha varianza se ajusta perfectamente al *training set* pero falla en el *test set* (sobreajuste). El objetivo es encontrar el punto óptimo entre ambos.

La SVM busca el hiperplano de máximo margen que separa las clases. En lugar de minimizar solo el error, maximiza la distancia entre el hiperplano y los puntos más cercanos de cada clase, que son los *vectores de soporte* [23]. Cuando las clases no son separables linealmente, la SVM proyecta los datos a un espacio de mayor dimensión usando una función kernel. El kernel RBF implementa el producto escalar en un espacio de dimensión infinita sin calcular explícitamente la proyección:

$$K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2} \quad (5.4)$$

donde $\gamma = 1/(\text{n_features} \cdot \text{Var}(\mathbf{X}))$. El parámetro $C = 1,0$ controla el margen blando (*soft margin*), así que permite algunos errores en entrenamiento a cambio de un margen más amplio.

Como la SVM de scikit-learn no produce probabilidades directamente, se aplica la calibración de Platt (CalibratedClassifierCV) con validación cruzada. Esta entrena una regresión logística sobre las puntuaciones para mapearlas a $p(\text{spoof}) \in [0, 1]$, que es lo que necesitan EER y minDCF.

5.2.3. XGBoost

El *gradient boosting* (potenciado de gradiente) construye un grupo de árboles de decisión de forma secuencial, donde cada nuevo árbol aprende a corregir los residuos del anterior. Un árbol de decisión divide los datos por umbrales de características hasta que las muestras de cada hoja son lo más homogéneas posible. A diferencia de la regresión lineal, los árboles de decisión no necesitan que los datos estén escalados o normalizados, y tienen la capacidad de detectar relaciones no lineales complejas entre las variables. [23].

En XGBoost cada árbol minimiza una función objetivo regularizada. Para decidir si vale la pena dividir un nodo, el algoritmo calcula la ganancia evaluando cuánto disminuyen los residuos antes y después de la separación. La cobertura (*cover*) de cada hoja es:

$$\text{Cover} = \sum_i [p_i \cdot (1 - p_i)] + \lambda \quad (5.5)$$

donde p_i es la probabilidad predicha por el modelo actual para la muestra i y λ es la regularización L2. Esta expresión penaliza hojas con pocas muestras y árboles con pesos de hoja grandes. La configuración del proyecto utiliza `n_estimators=300`, `max_depth=6` (árboles poco profundos que evitan memorizar el ruido), `learning_rate=0.1` (encogimiento de la contribución de cada árbol), `subsample=0.8` y `colsample_bytree=0.8` (submuestreo estocástico de filas y columnas para decorrelacionar los árboles), y `reg_alpha=0.1` (regularización L1 adicional). El método de histograma (`tree_method='hist'`) reduce el tiempo de entrenamiento agrupando los valores de cada característica en contenedores.

5.3. Redes neuronales convolucionales

5.3.1. Fundamentos teóricos y principios de la convolución

Una red neuronal artificial está formada por capas de neuronas. Cada neurona aplica una transformación a sus entradas y pasa el resultado por una función de activación [24]:

$$\mathbf{a}^{(l+1)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l)} + \mathbf{b}^{(l)}) \quad (5.6)$$

La función sigmoide era la activación estándar, pero hoy está prácticamente reemplazada por la ReLU que evita el problema del desvanecimiento de gradiente:

$$\text{ReLU}(x) = \max(0, x) \quad (5.7)$$

Su derivada es constante e igual a 1 para $x > 0$, lo que permite que el gradiente fluya sin disminuir durante la retropropagación. El entrenamiento se basa en minimizar la función de coste desplazándose en dirección contraria al gradiente mediante el SGD, que es la que más reduce el coste. El optimizador Adam es una variante del SGD que mantiene momentos de primer y segundo orden del gradiente para ajustar el *learning rate*, convergiendo más rápido en superficies de coste irregulares [22].

La función de pérdida para la clasificación binaria es la entropía cruzada binaria con logits:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] \quad (5.8)$$

Se usa la versión “con logits” (BCEWithLogitsLoss) porque combina la sigmoide con la entropía cruzada mediante el truco *log-sum-exp*, obteniendo un resultado estable que evita desbordamientos al elevar e^x con x muy grande.

La operación de convolución aplica filtros locales en lugar de pesos conectados. Un filtro F de 3×3 se desplaza sobre la entrada y en cada posición calcula un producto escalar:

$$(S * F)(i, j) = \sum_p \sum_q S(i + p, j + q) \cdot F(p, q) \quad (5.9)$$

El resultado es un mapa de características que indica dónde aparece el patrón que detecta el filtro. La ventaja de este esquema es la ecuanimidad traslacional, ya que si un artefacto sintético aparece en los primeros o últimos segundos, el mismo filtro lo detecta porque los pesos se comparten en toda la dimensión temporal [24]. Después de cada convolución se aplica una *BatchNorm* que re-normaliza las activaciones de cada canal usando la media y varianza del lote:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B^{(k)}}{\sqrt{(\sigma_B^{(k)})^2 + \varepsilon}} \quad (5.10)$$

Esto estabiliza la distribución de activaciones entre capas, permite tasas de aprendizaje más altas y actúa como regularizador. Posteriormente tras la ReLU se aplica el *MaxPooling*. Un `MaxPool2d(2 × 2)`

selecciona el valor máximo de cada región reduciendo a la mitad las dimensiones espaciales. Esto aporta invariancia local y reduce el coste de las capas siguientes. Por último se aplica el Dropout, donde $\text{Dropout}(p=0.3)$ apaga aleatoriamente el 30 % de las neuronas en cada paso de entrenamiento. Este proceso impide que las neuronas se adapten unas a otras, logrando el mismo efecto que si se entrenara un conjunto de subredes a la vez.

5.3.2. Diseño y configuración de la red convolucional base

La arquitectura base de la CNN son cinco bloques convolucionales con un número de filtros que se dobla progresivamente: $1 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. Cada bloque sigue el patrón:

$$\text{Conv2d}(3 \times 3) \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2 \times 2) \quad (5.11)$$

La entrada es un tensor de forma $(B, 1, 128, 300)$ (lote, canal único en escala de grises, bandas de frecuencia y tramas). Cada MaxPool2d divide las dimensiones espaciales por 2. Tras cinco bloques, un $\text{AdaptiveAvgPool2d}(4, 8)$ comprime cualquier tamaño a una representación fija de $256 \times 4 \times 8$ que pasa por una capa con Dropout y una capa de salida de un único logit.

Los bloques iniciales aprenden bordes y texturas locales como patrones armónicos o fronteras entre fonemas. Los intermedios detectan estructuras de mayor escala como bandas de ruido de vocoder y discontinuidades entre frames. Los profundos agrupan la evidencia que permite la decisión final [24]. En la Figura 5.3 se detalla todo este flujo de datos desde el espectrograma de entrada hasta la clasificación, mostrando los bloques convolucionales ($16 \rightarrow 256$ canales), el *pooling* espacial, la agregación global y la etapa final con *Dropout*.

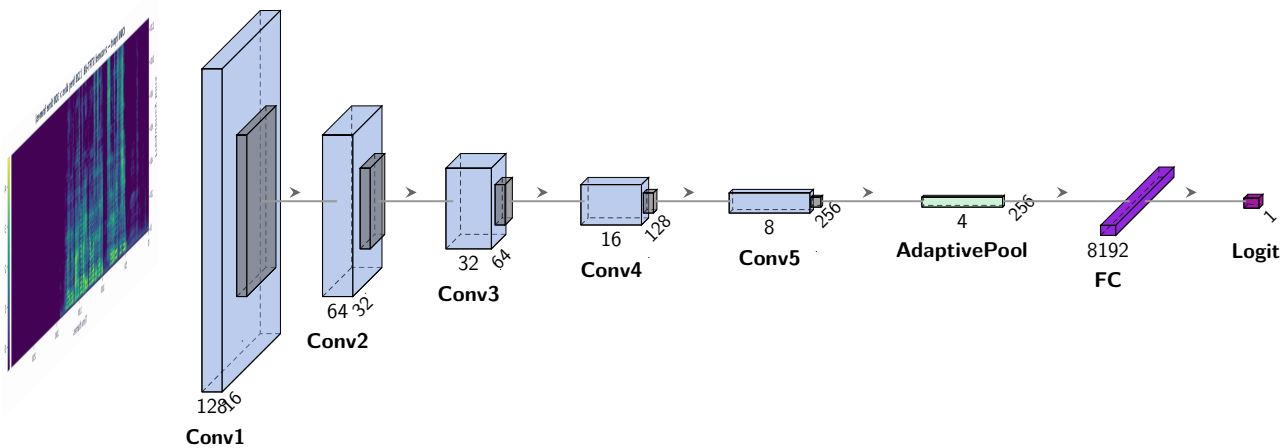


Figura 5.3: Arquitectura de la red CNN.

5.3.3. Bloques de atención Squeeze-and-Excitation

El bloque Squeeze-and-Excitation [14] aprende a reasignar el peso de cada mapa de características en función de su relevancia para la decisión. El mecanismo tiene dos pasos. El Squeeze aplica un *global average pooling* para reducir cada canal a un único escalar, y el Excitation procesa el vector

$\mathbf{z}$ a través de un pequeño MLP y activación final sigmoide, produciendo pesos en $[0, 1]$ para cada canal:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j) \quad (5.12)$$

$$\mathbf{s} = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{z})) \quad (5.13)$$

Los mapas de características originales se multiplican elemento a elemento por esos pesos, amplificando los canales importantes y eliminando los irrelevantes:

$$\tilde{\mathbf{u}}_c = s_c \cdot \mathbf{u}_c \quad (5.14)$$

Este bloque puede aprender a suprimir bandas de frecuencia contaminadas por el codec telefónico y amplificar la banda de alta frecuencia donde los vocoders dejan sus artefactos. [14].

5.3.4. Redes residuales con atención por canal

En 2015, los investigadores de Microsoft observaron que añadir capas a una red ya previamente entrenada empeoraba su rendimiento en lugar de mejorarlo [13]. El problema es que el gradiente se degrada al retropropagarse por muchas capas no lineales. La solución es la conexión residual, donde en lugar de aprender directamente la función $\mathcal{H}(x)$, el bloque aprende el residuo $\mathcal{F}(x) = \mathcal{H}(x) - x$, y la salida final es la suma $\mathcal{H}(x) = \mathcal{F}(x) + x$. Si la transformación óptima es $(\mathcal{H}(x) = x)$, solo hace falta que $\mathcal{F}(x) \rightarrow 0$, lo que es mucho más fácil de aprender que reproducir x con pesos no nulos. Esto resuelve el problema de degradación y permite entrenar redes mucho más profundas [13]. En la Figura 5.4 se presenta el esquema del flujo de identidad x y el residuo $\mathcal{F}(x)$.

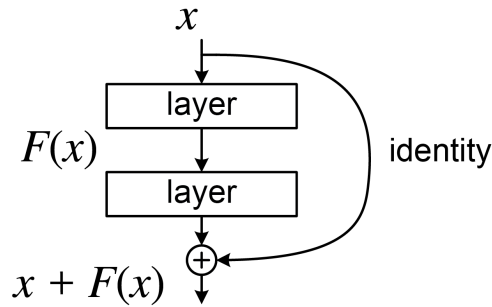


Figura 5.4: Bloque de conexión residual.

Esto da lugar a las redes ResNet_SE, que en este proyecto se implementan con cuatro bloques residuales (`_ResBlock`) con conexión de salto y bloque SE integrado. El patrón de cada bloque es:

$$\text{Conv}(3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(3 \times 3) \rightarrow \text{BN} \quad (5.15)$$

$$\text{salida} = \text{ReLU}(\mathcal{F}(x) + \text{skip}(x)) \rightarrow \text{SE} \rightarrow \text{MaxPool}(2 \times 2) \quad (5.16)$$

Para compensar la diferencia de canales de entrada y salida ($1 \rightarrow 32$ en el primer bloque), se usa una convolución 1×1 en la rama de salto para ajustar las dimensiones. La progresión de canales es $1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 128$. La combinación de las conexiones residuales, el BatchNorm y los bloques SE hace que este modelo sea más robusto al desplazamiento de dominio entre 2019 y 2021 que la CNN de 5 bloques base.

ResNeXt_SE extiende ResNet_SE introduciendo el concepto de la cardinalidad. Las convoluciones de 3×3 dentro de cada bloque residual se dividen en 32 grupos independientes [24]. En lugar de aprender un único conjunto de filtros sobre todos los canales, la red aprende 32 conjuntos de filtros sobre subconjuntos de canales independientes que se concatenan a la salida. Esto mejora la eficiencia, ya que al procesar subgrupos de canales en paralelo, los filtros son más pequeños sin reducir la capacidad expresiva global, y la regularización, porque descorrelaciona los caminos de características y mejora la generalización [24]. En los resultados experimentales (Capítulo 6), la ResNeXt_SE es el modelo más estable, con una desviación típica del EER de solo 0.04 % frente a 7.5 % de las otras arquitecturas.

5.4. Redes Neuronales Convolucionales Recurrentes

Las CNN ven el espectrograma como una imagen, detectando patrones pero sin tener noción de orden entre frames, sin embargo, el habla tiene estructura temporal. Los *deepfakes* a veces no se delatan por un artefacto puntual, sino por una progresión temporal inusual entre frames [8]. Las RNN tratan esto manteniendo un estado oculto $\mathbf{h}_t$ que se actualiza con cada nuevo frame:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}) \quad (5.17)$$

El problema de las RNN es que el gradiente decrece exponencialmente al retropropagar por muchos pasos de tiempo, lo que hace que la red olvide el contexto lejano. La LSTM resuelve esto con una celda de estado $\mathbf{c}_t$ controlada por tres puertas que deciden qué información se conserva, qué información nueva entra y qué información sale hacia la siguiente capa:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{puerta de olvido}) \quad (5.18)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{puerta de entrada}) \quad (5.19)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (5.20)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (5.21)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{puerta de salida}) \quad (5.22)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (5.23)$$

La GRU es una simplificación de la LSTM que fusiona las puertas de olvido y entrada en una única puerta de actualización, con un rendimiento similar pero menor coste. El proyecto usa GRU bidireccional, por lo que se procesa la secuencia hacia adelante y hacia atrás para que cada frame tenga contexto pasado y futuro [8].

La CRNN combina el extractor de características de la CNN de 5 bloques con una GRU bidireccional sobre el eje temporal [25].

El extractor convolucional procesa el espectrograma $\in (B, 1, 128, 300)$ y produce un mapa de características $\in (B, 256, 4, T')$ donde T' es la dimensión temporal residual. Después, un `AdaptiveAvgPool2d(4, None)` comprime el eje de frecuencia a 4 canales preservando el eje temporal, a diferencia de la CNN que lo colapsa completamente. Los mapas se reorganizan y se obtiene una secuencia de vectores por frame:

$$(B, 256, 4, T') \xrightarrow{\text{permute+reshape}} (B, T', 1024) \quad (5.24)$$

Esta secuencia entra en la GRU bidireccional y produce $(B, T', 256)$. El promedio sobre el eje temporal da un vector de 256 dimensiones por audio, que pasa por Dropout y una capa lineal para producir el logit final.

La ventaja de esta arquitectura sobre la CNN es que no descarta la secuencia temporal. Esto por ejemplo le permite detectar si la energía en una banda de frecuencia específica sube y baja con un patrón raro.

5.5. Aprendizaje autosupervisado y el modelo Wav2Vec 2.0

Wav2Vec 2.0 pertenece a otra familia de detectores que son los SSL, que aprenden representaciones de audio directamente de la forma de onda cruda, sin etiquetas [8].

Durante el entrenamiento, el modelo oculta partes de la señal y aprende a reconstruirlas eligiendo la opción correcta de entre un grupo de opciones falsas (aprendizaje por contraste). Esto entrena 12 capas *Transformer* con atención multi-cabeza sobre la forma de onda a 16 kHz sin necesitar las etiquetas *bonafide/spoof*. En este proyecto, el modelo se *fine-tunea* añadiendo una cabeza lineal `Linear(768, 2)` sobre la media temporal de los estados ocultos. Antes de procesar el audio, el sistema ajusta cada grabación para que tenga un volumen y una escala estándar (media cero y varianza unitaria). Esto asegura que los datos sean idénticos a los que espera el modelo de HuggingFace para funcionar correctamente.

Un detalle de implementación relevante es la calibración de temperatura. El *fine-tuning* produce logits muy seguros ($\approx \pm 8$) que saturan la sigmoide casi a 0 o 1. Para suavizar las probabilidades los logits se dividen por $T = 2,0$ antes del softmax:

$$p(\text{spoof}) = \text{softmax}\left(\frac{\text{logits}}{T}\right)_1 \quad (5.25)$$

5.6. Entrenamiento y estrategia experimental

Durante el entrenamiento se aplica SpecAugment sobre el conjunto de entrenamiento, ocultando aleatoriamente n bandas de frecuencia y m ventanas temporales en cada espectrograma para evitar que la red dependa de zonas concretas del espectro y mejorar la generalización fuera del dataset de 2019. El optimizador es Adam ($\text{lr}=0.00100$ para las CNN básicas y $\text{lr}=0.00050$ para ResNet+SE, ResNeXt+SE y CRNN), con `batch_size=32` (16 para CRNN), `dropout=0.300` y una parada temprana con paciencia de 5 a 7 épocas.

La tasa de aprendizaje no es fija durante toda la ejecución. Un planificador ReduceLROnPlateau la reduce a la mitad (factor=0.500) cuando la pérdida de validación no mejora durante la mitad de la paciencia configurada para la parada temprana. Ambos mecanismos comparten la misma señal, de modo que, antes de detener el entrenamiento por completo, el sistema intenta primero recuperar la mejora reduciendo el paso de actualización.

El entrenamiento en GPU usa precisión mixta, lo que acelera cada época entre 1.500 y 2.000 veces sin pérdida de precisión apreciable. El formato inicial, fp16, dio problemas con ResNet+SE ya que sus activaciones desbordaban, la pérdida de validación se volvía `inf/nan` sin dar ningún error visible, y el checkpoint nunca se actualizaba, así que el modelo final quedaba sin entrenar. La solución fue usar bfloat16 cuando la GPU lo soporta, ya que con 8 bits de exponente las activaciones no desbordan y tampoco hace falta el escalado de gradiente. Como garantía extra, ningún checkpoint se guarda si la pérdida de validación no es finita, y si ninguna época mejora, se devuelven los pesos de la última en vez de la inicialización aleatoria (Listado 5.2).

```
1 if math.isfinite(val_loss) and val_loss < best_val_loss - 1e-4:
2     best_val_loss = val_loss
3     epochs_no_impr = 0
4     best_state_dict = copy.deepcopy(model.state_dict())
5     ever_saved = True
6 else:
7     epochs_no_impr += 1
8     if epochs_no_impr >= patience:
9         if ever_saved:
10             model.load_state_dict(best_state_dict)
11         return time.perf_counter() - start, epoch, history
```

Listado 5.2: Guardado de checkpoint ante pérdida de validación no finita (src/pipeline.py).

En las arquitecturas más profundas (ResNet+SE, ResNeXt+SE) también se recorta la norma del gradiente (max_norm=5.000). Si un paso de entrenamiento produce un gradiente demasiado grande, se reduce antes de aplicarlo para que no desestabilice la red. Esta es una segunda barrera de estabilidad distinta de bf16, la cual evita que las activaciones desborden, y el recorte de gradiente evita que un paso de optimización brusco desestabilice la red.

Cada arquitectura se entrena con tres semillas (seeds=[42, 43, 44]). El leaderboard reporta la media de las tres y se conserva el checkpoint de la semilla con mejor pérdida de validación, que no siempre es el *dev* habitual. La función de entrenamiento admite un conjunto distinto, por ejemplo dejando fuera los ataques A05 y A06, para que la parada temprana y el planificador de tasa de aprendizaje premien la generalización a ataques no vistos en vez de memorizar los ya conocidos. En este último caso el *dev* original solo se usa para reportar la métrica final.

Para el desbalance de clases, los modelos clásicos usan `class_weight='balanced'` o `scale_pos_weight`, y las CNN hacen lo mismo de forma explícita con el `pos_weight` de BCEWithLogitsLoss, calculado en cada ejecución como el cociente bonafide/spoof del entrenamiento para penalizar más el error sobre la clase minoritaria.

Por último, para optimizar el coste computacional del experimento, los espectrogramas se cachean en disco para no recalcular la STFT en cada época, y como la entrada de la CNN tiene siempre el mismo tamaño (128x300), se activa el modo benchmark de cuDNN para que elija el algoritmo de convolución más rápido.

Capítulo 6

Configuración experimental y resultados

Para hacer una valoración de los resultados obtenidos, se describirá el entorno de ejecución y la interpretación de los datos que ha habido. Se descubrirá qué ocurre cuando los modelos entrenados en el corpus controlado de 2019 se enfrentan a condiciones que no vieron en entrenamiento como el canal telefónico (2021 LA) y los *deepfakes* capturados en condiciones reales (2021 DF).

6.1. Entorno de ejecución, evaluación y despliegue

El entrenamiento de todos los modelos se realizó en local sobre una máquina con GPU compatible con CUDA. Los modelos clásicos utilizaron exclusivamente la CPU, ya que scikit-learn y XGBoost no delegan en CUDA para los tamaños de datos que maneja este corpus, y el tiempo de entrenamiento se mide en segundos. Los modelos profundos se entrenaron con PyTorch sobre la GPU con precisión flotante de 32 bits. El modelo wav2vec 2.0 no se reentrenó desde cero, se partió del *checkpoint* público base de HuggingFace (Sara1708/deepfake-audio-wav2vec2) y se ajustó únicamente la cabeza de clasificación lineal sobre el corpus 2019 LA, descrito en el Apartado 5.5. El entorno de *software* fue Python 3.12, PyTorch 2.x, librosa, scikit-learn, XGBoost y HuggingFace Transformers. Los parámetros físicos de la señal y los hiperparámetros de cada arquitectura se centralizan en config/config.yaml sin hardcoding en el código.

Se evalúan cuatro particiones, como se describió en el Capítulo 4:

- Dev · 2019 LA: partición de desarrollo, comparte sistemas de ataque (A01–A06) con *train*. Útil para observar el comportamiento en distribución pero no representa la capacidad de generalización real.
- Eval · 2019 LA: 13 sistemas de ataque (A07–A19) nunca vistos durante el entrenamiento. Primera medida real de generalización.
- Eval · 2021 LA: los mismos tipos de ataque que 2019 pero con degradación de canal telefónico real.
- Eval · 2021 DF: *deepfakes in the wild*, mayor diversidad de sistemas generativos y condiciones acústicas no controladas. El escenario más exigente.

Las métricas son EER (%) y minDCF con los parámetros oficiales de ASVspoof 2019. Para los modelos CNN, cada arquitectura se entrena tres veces con semillas distintas y se reportan la media y la desviación estándar. La parada temprana usa la partición de validación de ataques no vistos (A05/A06 del conjunto de entrenamiento).

El frontend Streamlit se despliega en Streamlit Cloud en modo solo-CPU. En ese entorno, los modelos clásicos y las CNN cargan sus pesos (.joblib y .pth) directamente del repositorio. El checkpoint de wav2vec 2.0 (≈ 469 MB) excede el límite de 100 MB de GitHub y se descarga en el primer arranque desde el repositorio público de HuggingFace. Las métricas de latencia reportadas en el Apartado 6.3 son las de GPU local, no las de la nube.

6.2. Análisis comparativo de resultados por corpus

Las siguientes tablas recogen los EER (%) y minDCF de todos los sistemas evaluados, agrupados por familia de modelo. Para los sistemas profundos se incluye la desviación estándar sobre tres semillas ($\pm\sigma$). Los resultados de desarrollo (Tabla 6.1) muestran el comportamiento en distribución, los de evaluación (Tabla 6.2, Tabla 6.3, Tabla 6.4) miden la generalización real.

Tabla 6.1: Resultados en la partición de desarrollo (*Dev* · 2019 LA).

Tipo	Sistema	EER (%)	minDCF
SSL	wav2vec 2.0	0.710	0.056
CNN	5-Block CNN + SE	$0,820 \pm 0,080$	$0,290 \pm 0,082$
	ResNeXt+SE	$1,100 \pm 0,040$	$0,244 \pm 0,039$
	5-Block CNN	$4,840 \pm 7,100$	$0,470 \pm 0,459$
	CRNN	$5,770 \pm 7,680$	$0,713 \pm 0,324$
	ResNet+SE	$9,170 \pm 7,510$	$0,750 \pm 0,431$
Clásico	LFCC + SVM RBF	2.430	0.469
	LFCC + XGBoost	3.610	0.427
	LFCC + LR	4.100	0.782
	CQCC + SVM RBF	6.360	0.850
	CQCC + XGBoost	7.650	0.841
	MFCC + SVM RBF	11.420	0.916

6.2.1. Generalización consistente en modelos wav2vec 2.0

La diferencia más llamativa de las tablas es la que separa a wav2vec 2.0 del resto en los tres corpus de evaluación real. En 2019 LA eval su EER (4.960 %) es menos de la mitad del siguiente mejor modelo. en 2021 LA y 2021 DF se mantiene en torno al 13 %, mientras las CNN se degradan hasta el 19–28 % y los modelos clásicos hasta el 22–42 %. Esto no se aprecia en *dev* donde wav2vec llega al 0.710 % y las CNN a 0.820–9.170 %, pero ese margen de diferencia es engañoso porque *dev* comparte ataques con *train*.

Tabla 6.2: Resultados en la partición de evaluación (*Eval* · 2019 LA).

Tipo	Sistema	EER (%)	minDCF
SSL	wav2vec 2.0	4.960	0.674
CNN	ResNeXt+SE	11,400 ± 0,800	0,984 ± 0,019
	5-Block CNN	11,250 ± 3,260	0,991 ± 0,002
	5-Block CNN + SE	11,260 ± 0,210	0,996 ± 0,000
	CRNN	13,400 ± 2,960	0,995 ± 0,004
	ResNet+SE	13,400 ± 3,620	0,997 ± 0,004
Clásico	CQCC + SVM RBF	12.840	0.985
	CQCC + XGBoost	13.800	0.921
	LFCC + XGBoost	15.140	0.994
	MFCC + SVM RBF	15.350	0.999
	LFCC + SVM RBF	15.640	0.998

Tabla 6.3: Resultados en la partición de evaluación (*Eval* · 2021 LA).

Tipo	Sistema	EER (%)	minDCF
SSL	wav2vec 2.0	13.040	0.811
CNN	CRNN	19,060 ± 1,180	0,997 ± 0,002
	ResNet+SE	19,720 ± 1,280	0,997 ± 0,002
	5-Block CNN	19,740 ± 1,430	0,998 ± 0,004
	ResNeXt+SE	20,020 ± 1,030	0,988 ± 0,009
	5-Block CNN + SE	20,440 ± 1,940	0,996 ± 0,004
Clásico	MFCC + SVM RBF	22.010	0.999
	LFCC + SVM RBF	33.960	0.999
	CQCC + SVM RBF	39.250	0.989

Tabla 6.4: Resultados en la partición de evaluación (*Eval* · 2021 DF).

Tipo	Sistema	EER (%)	minDCF
SSL	wav2vec 2.0	13.510	0.851
	ResNeXt+SE	$25,090 \pm 0,550$	$0,984 \pm 0,014$
	CRNN	$26,100 \pm 0,220$	$0,990 \pm 0,010$
CNN	5-Block CNN + SE	$26,210 \pm 1,000$	$0,985 \pm 0,014$
	5-Block CNN	$27,140 \pm 1,670$	$0,997 \pm 0,003$
	ResNet+SE	$27,930 \pm 0,700$	$0,999 \pm 0,002$
Clásico	MFCC + SVM RBF	28.660	1.000
	CQCC + SVM RBF	41.890	0.983
	LFCC + SVM RBF	39.860	0.987

La explicación de la robustez bajo desplazamiento de dominio es que wav2vec 2.0 aprende representaciones sobre cientos de horas de voz real mediante aprendizaje autosupervisado antes de ver una sola etiqueta de *spoof* [8]. El ajuste sobre 2019 LA añade la discriminación spoof/bonafide encima de una representación de habla ya generalizable. Las CNN aprenden esa representación de cero y quedan más ligadas a los artefactos específicos de los sistemas A01–A06.

6.2.2. Rendimiento de arquitecturas convolucionales y clásicas

Las altas desviaciones estándar de algunos modelos en *dev* (5-Block CNN: ± 7.100 ; CRNN: ± 7.680 ; ResNet+SE: ± 7.510) muestran que distintas semillas pueden terminar en puntos muy distintos cuando el conjunto de validación está saturado. El criterio de parada temprana sobre ataques no vistos (A05/A06) soluciona este problema pero no lo elimina completamente en el conjunto de *dev*.

En las particiones de evaluación, las desviaciones estándar se reducen bastante (0.220–3.620 %), lo que indica que la varianza en condiciones reales se puede manejar. El modelo más estable es ResNeXt+SE, con una desviación estándar de EER de ± 0.040 en *dev* y ± 0.800 en 2019 LA eval. Las convoluciones agrupadas (*grouped convolutions*, cardinality = 32) lo regulan al forzar a cada grupo de filtros a aprender patrones independientes.

En los modelos clásicos las características de alta frecuencia son las más discriminantes para la detección de *deepfakes*. En 2019 LA *dev*, la LFCC alcanza EER del 2.430 % con SVM frente al 11.420 % de la MFCC con el mismo clasificador. La explicación está en el diseño del banco de filtros, ya que la escala mel comprime la frecuencia por encima de 1 kHz, que es donde los vocoders dejan artefactos de cuantización, mientras que la LFCC mantiene la resolución y las captura.

Sin embargo, en 2021 LA la LFCC+SVM pasa de 2.430 % en *dev* a 33.960 %, mientras que la MFCC+SVM solo sube a 22.010 %. Esto ocurre porque el canal telefónico elimina la mayoría de información por encima de 3.4 kHz, borrando los artefactos y dejando la señal en una región donde la escala mel ya no es un problema. Esto demuestra que la superioridad de un modelo frente a otro no es absoluta, sino que se encuentra en la dependencia del canal.

6.2.3. Variación del rendimiento entre conjuntos *dev* y *eval*

La distancia entre los resultados en *dev* y en *eval* es el indicador de que existe sobreajuste con el conjunto de entrenamiento. Para la 5-Block CNN+SE, la EER pasa de 0.820 % en *dev* a 11.260 % en 2019 LA *eval*, incluso siendo los mismos sistemas de ataque. Para las CNNs esto es puramente consecuencia del diseño, ya que el modelo nunca se optimizó para generalizar a ataques no vistos en *dev* sino en la partición A05/A06.

6.3. Eficiencia computacional

Además de la calidad de detección, el coste de cada sistema es relevante para su viabilidad en producción. Se midieron tres componentes de tiempo:

- Tiempo de entrenamiento: El coste computacional inicial, proceso que se realiza una única vez.
- Tiempo de extracción de características: La duración del preprocesamiento de la señal de audio aplicada a cada clip.
- Latencia de inferencia: El tiempo que tarda el modelo en completar el paso para generar una predicción.

Los tiempos se miden en la máquina local con GPU y las latencias de extracción e inferencia son medias sobre todos los corpus evaluados.

Los modelos clásicos tienen una latencia total de 0.5 a 0.7 ms por clip, muy por debajo de todos los modelos profundos. Sin embargo, su calidad de detección en condiciones reales (EER > 22 % en 2021) los hace insuficientes para un despliegue donde se esperan *deepfakes* de calidad producidos con sistemas diferentes a los de entrenamiento.

Las CNN tienen un equilibrio más razonable: 3 a 7 ms de latencia total y entrenamientos de entre 21 minutos (5-Block CNN) y 4.4 horas (ResNeXt+SE). El hecho de que CNN+SE y CRNN sean más rápidas en extracción que la CNN base es motivo de la caché de espectrogramas en *benchmark*, ya que cuando se lanza la comparación completa, el espectrograma ya está calculado para las variantes que se evalúan después de la CNN base. En producción, la extracción es la misma para todas las CNN, es decir, una transformada STFT de ventana Hann de 1024 puntos con *hop* de 512 muestras.

wav2vec 2.0 es el sistema más lento (24.613 ms por clip) y a la vez el que tiene el mejor EER medio sobre todos los corpus. En un escenario en tiempo real, latencias de hasta 25 ms son perfectamente aceptables, ya que equivalen a procesar 40 clips por segundo. El coste real de wav2vec es el de inferencia pura, y dado que no requiere extracción de características, la forma de onda de 16 kHz entra directamente al modelo.

Tabla 6.5: Eficiencia computacional de los sistemas.

Tipo	Sistema	Entren. (s)	Extrac. (ms/clip)	Infer. (ms/clip)
Clásico	LFCC + XGBoost	8.920	0.586	0.001
	LFCC + SVM RBF	0.330	0.586	0.024
	CQCC + SVM RBF	0.700	0.530	0.040
	MFCC + SVM RBF	0.920	0.557	0.056
CNN	5-Block CNN	1 270.880	6.918	0.314
	CRNN	1 676.800	2.907	0.361
	5-Block CNN + SE	2 079.470	2.861	0.353
	ResNet+SE	4 951.620	2.724	1.357
	ResNeXt+SE	15 973.100	3.065	1.095
SSL	wav2vec 2.0	—	—	24.613

Tabla 6.6: Latencia total y rendimiento por sistema.

Tipo	Sistema	Latencia total (ms/clip)	EER medio (%) — todos los corpus
Clásico	LFCC + XGBoost	0.587	24.008
	LFCC + SVM RBF	0.610	22.973
	CQCC + SVM RBF	0.570	25.085
	MFCC + SVM RBF	0.613	19.360
CNN	5-Block CNN	7.232	15.743
	CRNN	3.268	16.083
	5-Block CNN + SE	3.214	14.683
	ResNet+SE	4.081	17.555
	ResNeXt+SE	4.160	14.403
SSL	wav2vec 2.0	24.613	8.055

6.4. Robustez ante el desplazamiento de dominio

El resultado más relevante del experimento es la brecha que se abre entre los modelos cuando se pasa a los corpus de 2021. La columna de "salto" de la Tabla 6.7 muestra cuánto empeora el rendimiento de cada modelo al pasar del conjunto de datos de entrenamiento a un entorno más real, por lo que indica qué modelos son menos estables ante cambios de escenario. Por ejemplo, el sistema CQCC con SVM tiene un error bajo de 12.840 % en 2019, pero sube mucho en 2021, llegando al 41.890 %. Esto demuestra que realmente no son buenos detectando *deepfakes*, sino que aprendieron a reconocer patrones específicos de los ataques que no aparecen en grabaciones reales.

Las CNN sufren un salto de 13 a 15 puntos, lo que indica que los patrones aprendidos sobre espectrogramas STFT generalizan algo mejor que los modelos clásicos, pero siguen siendo sensibles al cambio de distribución. El menor salto entre todas las arquitecturas CNN corresponde al CRNN (12.700 pp), debido a que la capa GRU bidireccional captura patrones temporales de más largo alcance que son menos dependientes de artefactos locales.

wav2vec 2.0 es el único sistema que se mantiene por debajo del 14 % de EER en ambos corpus de 2021, con un salto de solo 8.550. Su ventaja es su representación interna del habla aprendida auto-supervisadamente, que no está anclada a los artefactos de una familia concreta de vocoders.

En conjunto, los resultados confirman que la detección de *deepfakes* de audio es un problema de generalización más que de clasificación. Cualquier sistema puede alcanzar errores bajos en un corpus controlado entrenando sobre los mismos tipos de ataque que evaluará, pero el reto real es mantener ese rendimiento cuando los ataques y las condiciones de canal cambian. La estrategia más eficaz sería entrenar al modelo para entender el habla de forma general antes de enseñarle a detectar voces falsas, lo que evitaría que el sistema dependa de los procesos específicos de cada método de síntesis.

Tabla 6.7: Evolución del EER (%) a través de los corpus de evaluación.

Sistema	Dev 2019	Eval 2019 LA	Eval 2021 LA	Eval 2021 DF	Salto (pp)
wav2vec 2.0	0.710	4.960	13.040	13.510	+8.550
ResNeXt+SE	1.100	11.400	20.020	25.090	+13.690
5-Block CNN+SE	0.820	11.260	20.440	26.210	+14.950
CRNN	5.770	13.400	19.060	26.100	+12.700
ResNet+SE	9.170	13.400	19.720	27.930	+14.530
LFCC + XGBoost	3.610	15.140	42.560	34.720	+19.580
CQCC + SVM RBF	6.360	12.840	39.250	41.890	+29.050

La Tabla 6.7 resume la evolución del EER de cada sistema a través de los cuatro escenarios de evaluación; la Figura 6.1 representa esos mismos datos de forma visual para hacer más evidente la brecha de generalización entre familias de modelos.

La gráfica hace visible de un vistazo lo que se muestra en cifras. Wav2vec 2.0 (línea verde continua) apenas se despegue de la base al pasar de 2019 a 2021, mientras que los modelos clásicos basados en LFCC y CQCC (líneas discontinuas) sufren la pendiente más pronunciada, y las arquitecturas CNN ocupan una posición intermedia, con ResNeXt+SE como la más estable dentro de su familia.

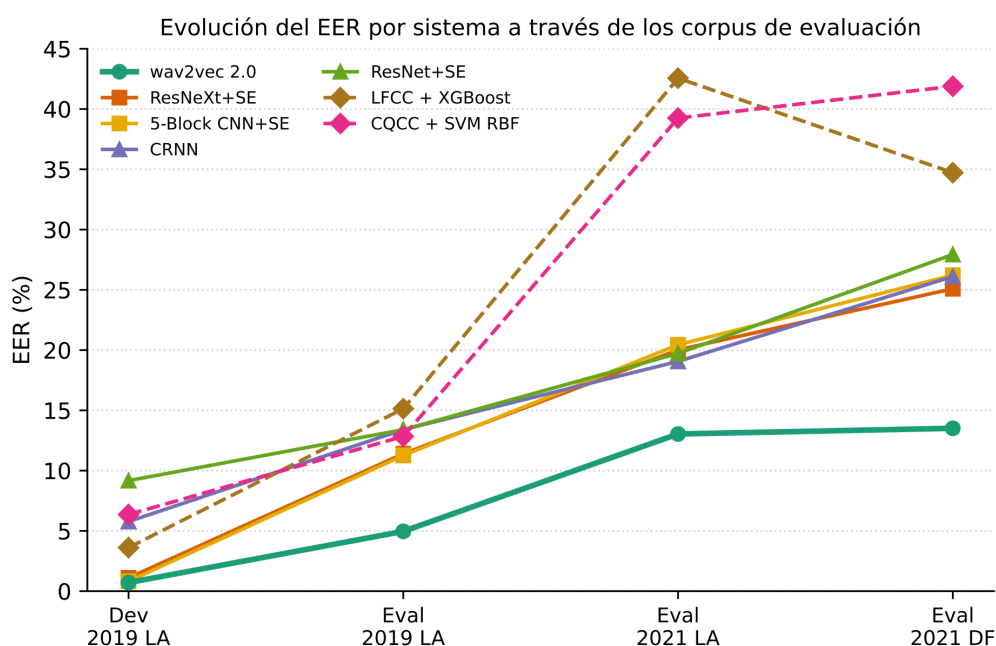


Figura 6.1: Evolución del EER por modelo a través de los corpus de evaluación.

Puesto en relación con el Apartado 6.3, este resultado añade un matiz práctico que conviene no perder de vista: el sistema con mejor generalización no es el más barato de ejecutar. Wav2vec 2.0 necesita 24.613 ms por clip frente a los 3–7 ms de las CNN o el menos de 1 ms de los modelos clásicos, de modo que la elección de arquitectura para un despliegue real depende de si prima la fiabilidad ante ataques desconocidos o la capacidad de respuesta en tiempo real. En conjunto, los resultados de este capítulo constituyen la base empírica sobre la que se apoyan las conclusiones y las líneas futuras que se desarrollan en el Capítulo 7.

Capítulo 7

Conclusiones y líneas futuras

7.1. Conclusiones

En conclusión, wav2vec 2.0 consigue un EER del 4.960 % en 2019 LA eval y lo mantiene por debajo del 14 % en los corpus de 2021 que nunca vio en entrenamiento. Ningún otro modelo se acerca a esas cifras en el *split* de evaluación. Esta ventaja proviene de su representación del habla, aprendida a partir de cientos de horas de voz real y no anclada a los artefactos de ninguna familia concreta de vocoders.

Frente a esto, las cinco arquitecturas construidas sobre espectrogramas STFT muestran una vulnerabilidad clara al sobreajuste por artefactos de síntesis: suben entre 13 y 15 puntos porcentuales de EER al pasar de 2019 a 2021, lo que indica que aprendieron patrones ligados a los sistemas de síntesis de entrenamiento más que propiedades generalizables del habla sintética. Dentro de esta familia, ResNeXt+SE es la excepción más robusta, ya que sus convoluciones agrupadas actúan como regularizador implícito y logran una desviación estándar entre semillas de apenas ± 0.040 , frente al rango de ± 7.100 – 7.680 del resto de arquitecturas.

Los modelos clásicos, por su parte, resultan inviables ante la variabilidad del dominio. Las combinaciones de características fijas (LFCC, CQCC, MFCC) con clasificadores estadísticos alcanzan un minDCF cercano a 1.000 en la evaluación real. Su latencia, inferior a 0.700 ms/clip, es su único argumento a favor, pero no compensa la pérdida de calidad frente al resto de sistemas.

Estos resultados evidencian además la necesidad de evaluaciones multi-corpus para evitar el sesgo de desarrollo: evaluar únicamente sobre el conjunto *dev* de 2019 habría invertido el ranking, con la CNN+SE alcanzando un 0.820 % de EER frente al 0.710 % de wav2vec, de modo que son los corpus de 2021 los que revelan la evaluación real.

Los ataques de ASVspoof 2019 se generaron con los mejores sistemas de síntesis disponibles en aquel momento, pero desde entonces la generación de audio falso ha cambiado significativamente. Modelos como VALL-E y sus sucesores permiten ya la clonación de voz en cero disparos (*zero-shot voice cloning*) a partir de tres segundos de audio, y los sistemas basados en difusión han eliminado la mayoría de los artefactos espectrales que los detectores entrenados en 2019 aprendieron a identificar. Las plataformas comerciales ponen esa capacidad al alcance de cualquier usuario sin necesidad de conocimientos técnicos, e incluso existen herramientas de código abierto que permiten la conversión de voz en tiempo real con latencias inferiores a 200 ms.

Esta dinámica revela una desventaja defensiva estructural. Mejorar la calidad de un sistema generativo mejora automáticamente su capacidad para evadir detectores, mientras que mejorar un detector exige acceso previo a muestras de los nuevos sistemas de ataque, por lo que la defensa siempre va un paso por detrás. El EER del 13.510 % de wav2vec 2.0 en 2021 DF significa que, con la tecnología

de 2021, uno de cada ocho *deepfakes* no llegaría a detectarse. En definitiva, esto demuestra que el problema está lejos de encontrar una solución definitiva.

7.2. Líneas futuras

Para reducir la ventaja que tienen los sistemas de generación frente a los métodos de detección, este trabajo apunta a diferentes opciones. La primera pasa por modelos fundacionales de mayor capacidad: comparar distintos *backbones* SSL como WavLM [26], HuBERT [27] o wav2vec 2.0 *large* bajo el mismo protocolo multi-corpus permitiría comprobar si una mayor escala de preentrenamiento mejora la robustez ante el desplazamiento de dominio. Por otro lado, se propone validar el sistema sobre nuevos estándares como ASVspoof 5, ampliando el banco de pruebas para evaluar los modelos frente a vectores de ataque más avanzados y comprobar si la ventaja de wav2vec 2.0 se mantiene ante los nuevos escenarios de evaluación y métricas.

Más allá de la evaluación, el aprendizaje continuo permitiría diseñar sistemas capaces de aprender nuevos tipos de ataque de forma incremental sin olvidar los que ya saben detectar (*catastrophic forgetting*), algo especialmente relevante dado que las amenazas cambian constantemente. La fusión multi-modal, combinando el detector de audio con la detección de *deepfake* visual (movimiento de labios, expresión facial), aportaría una decisión más estable frente a ataques que combinen ambas modalidades. Por último, la optimización para arquitecturas ligeras resulta necesaria porque wav2vec 2.0 requiere GPU para ofrecer latencias aceptables. Comprimir su representación en una arquitectura ejecutable en CPU abriría su uso en teléfonos, módems VoIP y equipos de grabación en campo.

En conclusión, la dirección que marcan estas líneas de investigación futuras coincide con los descubrimientos principales de este proyecto. El camino más eficaz y sostenible a largo plazo para identificar contenidos sintéticos consiste en modelar con la mayor precisión posible las características del habla humana real. Así, los sistemas de defensa mantendrán su capacidad de discriminación, lo que asegura que la solución sea inmune al desarrollo de nuevos algoritmos de ataque.

Capítulo 8

Conclusions and Future Research

8.1. Conclusions

To conclude, wav2vec 2.0 achieves an EER of 4.960 % on the 2019 LA eval set and keeps it below 14 % on the 2021 corpora, which it never saw during training. No other model comes close to these numbers on the evaluation split. This advantage comes from its speech representation, learned from hundreds of hours of real speech and not bound to the artifacts of any particular vocoder family.

By contrast, the five architectures built on STFT spectrograms show a clear vulnerability to overfitting on synthesis artifacts: their EER rises by 13 to 15 percentage points when moving from 2019 to 2021, indicating that they learned patterns tied to the training synthesis systems rather than properties that generalize across synthetic speech. Within this family, ResNeXt+SE is the most robust exception, since its grouped convolutions act as an implicit regulariser, achieving a standard deviation across seeds of only ± 0.040 , against a range of ± 7.100 – 7.680 for the rest of the architectures.

The classical models, in turn, turned out to be unviable under domain variability. Fixed feature combinations (LFCC, CQCC, MFCC) paired with statistical classifiers reach a minDCF close to 1.000 on the real evaluation set. Their latency, under 0.700 ms/clip, is their only point in favor, but it does not offset the drop in quality compared with the rest of the systems.

These results further highlight the need for multi-corpus evaluations to mitigate development bias: evaluating only on the 2019 *dev* set would have reversed the ranking, with CNN+SE reaching an EER of 0.820 % against wav2vec’s 0.710 %, which means it is the 2021 corpora that reveal the real-world evaluation.

The ASVspoof 2019 attacks were generated with the best synthesis systems available at the time, but fake audio generation has changed considerably since then. Models such as VALL-E and its successors now allow *zero-shot voice cloning* from as little as three seconds of audio, and diffusion-based systems have removed most of the spectral artifacts that detectors trained in 2019 learned to pick up on. Commercial platforms put this capability within reach of any user with no technical knowledge required, and there are even open-source tools that allow real-time voice conversion with latencies under 200 ms.

This dynamic reveals a structural defensive disadvantage. Improving a generative system’s quality automatically improves its ability to evade detectors, whereas improving a detector requires prior access to samples from the new attack systems, meaning defence will always lag one step behind. wav2vec 2.0’s EER of 13.510 % on 2021 DF means that, with 2021 technology, one deepfake out of every eight would go undetected. Ultimately, this shows that the problem is far from being definitively solved.

8.2. Future Research

To narrow the advantage that generation systems hold over detection methods, this work points to different options. The first involves larger-capacity foundation models: comparing different SSL *backbones* such as WavLM [26], HuBERT [27] or wav2vec 2.0 *large* under the same multi-corpus protocol would make it possible to check whether a larger pretraining scale improves robustness against domain shift. A second line would consist of validating the system against new standards such as ASVspoof 5, extending the test bed to evaluate the models against more advanced attack vectors and checking whether wav2vec 2.0's advantage holds up under new evaluation scenarios and metrics.

Beyond evaluation, continual learning would allow the design of systems capable of learning new attack types incrementally without forgetting the ones they already detect (*catastrophic forgetting*), which is especially relevant given that threats keep evolving. Multi-modal fusion, combining the audio detector with visual deepfake detection (lip movement, facial expression), would provide a more stable decision against attacks that combine both modalities. Finally, optimization for lightweight architectures is needed because wav2vec 2.0 requires a GPU to deliver acceptable latency. Compressing its representation into an architecture that can run on CPU would open up its use on phones, VoIP modems and field recording equipment.

In conclusion, the direction pointed to by these future research lines matches the main findings of this project. The most effective and sustainable long-term path to identifying synthetic content lies in modeling the characteristics of genuine human speech as precisely as possible. By doing so, defense systems will retain their discriminative power, ensuring the solution remains resilient to the development of new attack algorithms.

Capítulo 9

Presupuesto

Se expondrá el coste estimado del proyecto, desglosado en equipos, licencias y mano de obra. Dado que todo el *software* utilizado es de código abierto o gratuito, el coste de las herramientas es mínimo y la mayor parte del presupuesto corresponde al trabajo de desarrollo e investigación.

El único coste de hardware es la amortización parcial del ordenador utilizado para el entrenamiento de los modelos. La duración del proyecto se estima en aproximadamente 4 meses. Todo el *software* empleado: Python, PyTorch, scikit-learn, XGBoost, librosa, HuggingFace Transformers, Streamlit, VS Code y GitHub es de código abierto o dispone de plan gratuito suficiente para las necesidades del proyecto. Los corpus ASVspoof 2019 y 2021 también son de descarga pública y gratuita, tal y como se detalla en la Tabla 9.1.

Tabla 9.1: Presupuesto de equipos y licencias

Descripción	Cantidad	Coste (€)
Ordenador personal con GPU NVIDIA (coste total 1.400 €)	1	175
Python, PyTorch, scikit-learn, XGBoost, librosa, etc.	—	0
Entorno de desarrollo Visual Studio Code	1	0
Plataforma de despliegue Streamlit Cloud (plan Community)	1	0
HuggingFace Hub (plan gratuito)	1	0
GitHub y control de versiones (repositorio público)	1	0
Modelos de IA y asistencia al código (planes gratuitos)	—	0
Corpus ASVspoof 2019 y 2021 (descarga pública)	—	0
Subtotal de equipos y licencias		175

El trabajo de desarrollo contempla la revisión bibliográfica, el diseño e implementación del sistema de extracción de características, el entrenamiento y evaluación de los 21 modelos, y el desarrollo del *frontend* Streamlit. Se estima una dedicación total de unas 330 horas valoradas al precio de un desarrollador junior, cuyo desglose detallado por fases se muestra en la Tabla 9.2. El resumen económico global y el coste final del proyecto se recogen en la Tabla 9.3.

Tabla 9.2: Coste de mano de obra

Descripción	Horas	Coste (€)
Precio por hora (perfil junior en IA / Ingeniería de Software)	—	15
Revisión bibliográfica y estudio del estado del arte	60	900
Diseño e implementación del sistema (extracción, modelos, pipeline)	100	1.500
Entrenamiento, evaluación y análisis de resultados	30	450
Desarrollo del frontend Streamlit y despliegue	100	1.500
Redacción de la memoria	40	600
Total de horas	330	
Coste total de mano de obra		4.950

Tabla 9.3: Coste total del proyecto

Descripción	Coste total (€)
Subtotal de equipos y licencias	175
Coste total de mano de obra	4.950
Coste total del proyecto	5.125

Apéndice

Todo el trabajo desarrollado en este proyecto es de acceso público. Tanto el código fuente como la aplicación web están disponibles sin necesidad de registro y se mantienen en un estado funcional equivalente al descrito en la memoria.

Repositorio de código

El código fuente completo está disponible en GitHub bajo licencia MIT:

<https://github.com/Sampeerez/Deepfake-Audio-Detection>

El repositorio contiene todos los módulos descritos en la memoria: los extractores de características (`src/features.py`), las arquitecturas de modelos (`src/models.py`), el pipeline de entrenamiento y evaluación (`src/pipeline.py`), las métricas EER y minDCF (`src/metrics.py`) y el frontend Streamlit (`app.py` y `app_pages/`). Incluye además los pesos entrenados de los 15 modelos clásicos y las cinco arquitecturas profundas en la carpeta `models/`, el fichero `leaderboard.json` con los resultados completos de todos los corpus, y una suite de más de 100 tests automatizados que cubre métricas, extracción de características, modelos y carga de datos. Las instrucciones de instalación y ejecución en local están documentadas en el `README.md`.

Aplicación web

La demo interactiva está desplegada en Streamlit Cloud y es accesible desde cualquier navegador sin instalación ni registro:

<https://deepfake-audio-detection-tfg.streamlit.app>

La aplicación integra las páginas descritas en la memoria. El *Signal Explorer* permite visualizar la forma de onda y todas las representaciones espectrales (STFT-dB, entrada CNN, MFCC, LFCC, CQCC) de cualquier audio, con un modo de comparación *bonafide* vs. *spoof*. La página de *Benchmark* ofrece tres modos: lanzar el pipeline clásico sobre cualquier combinación de característica y clasificador, entrenar una CNN en directo con visualización de curvas de pérdida por época, y consultar la tabla comparativa completa de los 21 modelos con sus métricas de eficiencia. La página de *Detection Analysis* permite subir un audio propio y obtener el veredicto de detección mediante la fusión ponderada de wav2vec 2.0 y ResNeXt+SE, así como inspeccionar las distribuciones de puntuación, la curva ROC/DET y el umbral de decisión de cualquier modelo sobre cualquiera de los corpus de evaluación.

Bibliografía

Las siguientes referencias bibliográficas se presentan en orden de aparición en el texto.

- [1] Zhizheng Wu et al. «ASVspoof 2015: Automatic speaker verification spoofing and countermeasures challenge evaluation plan». En: *Training* 10.15 (2014), pág. 3750.
- [2] Massimiliano Todisco et al. «ASVspoof 2019: Future horizons in spoofed and fake audio detection». En: *arXiv preprint arXiv:1904.05441* (2019).
- [3] Junichi Yamagishi et al. «ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection». En: *arXiv preprint arXiv:2109.00537* (2021).
- [4] Md. Sahidullah, Tomi Kinnunen y Cemal Hanilçi. «A comparison of features for synthetic speech detection». En: *Interspeech 2015*. 2015, págs. 2087-2091. DOI: 10.21437/Interspeech.2015-472.
- [5] Tianxiang Chen et al. «Generalization of Audio Deepfake Detection.» En: *Odyssey*. 2020, págs. 132-137.
- [6] Hemlata Tak et al. «End-to-end anti-spoofing with rawnet2». En: *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2021, págs. 6369-6373.
- [7] Jee-weon Jung et al. «Aasist: Audio anti-spoofing using integrated spectro-temporal graph attention networks». En: *ICASSP 2022-2022 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE. 2022, págs. 6367-6371.
- [8] Alexei Baevski et al. «wav2vec 2.0: A framework for self-supervised learning of speech representations». En: *Advances in neural information processing systems* 33 (2020), págs. 12449-12460.
- [9] Nicolas M Müller et al. «Does audio deepfake detection generalize?» En: *arXiv preprint arXiv:2203.16263* (2022).
- [10] Hemlata Tak et al. «Rawboost: A raw data boosting and augmentation method applied to automatic speaker verification anti-spoofing». En: *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2022, págs. 6382-6386.
- [11] Jee-weon Jung et al. «SASV challenge 2022: A spoofing aware speaker verification challenge evaluation plan». En: *arXiv preprint arXiv:2201.10283* (2022).
- [12] Jiangyan Yi et al. «Add 2022: the first audio deep synthesis detection challenge». En: *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2022, págs. 9216-9220.
- [13] Kaiming He et al. «Deep residual learning for image recognition». En: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, págs. 770-778.
- [14] Jie Hu, Li Shen y Gang Sun. «Squeeze-and-excitation networks». En: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, págs. 7132-7141.

- [15] Saining Xie et al. «Aggregated residual transformations for deep neural networks». En: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, págs. 1492-1500.
- [16] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.
- [17] Ashish Vaswani et al. «Attention is all you need». En: *Advances in neural information processing systems* 30 (2017).
- [18] Lawrence Rabiner y Ronald Schafer. *Theory and applications of digital speech processing*. Prentice Hall Press, 2010.
- [19] Mallat Stephane. *A wavelet tour of signal processing*. 1999.
- [20] Karla Schäfer y Martin Steinebach. «MFCC vs. LFCC for Audio Deepfake Detection: The Role of Delta Features and Input Length». En: *2025 33rd European Signal Processing Conference (EUSIPCO)*. IEEE. 2025, págs. 576-580.
- [21] Tomi Kinnunen et al. «t-DCF: a detection cost function for the tandem assessment of spoofing countermeasures and automatic speaker verification». En: *arXiv preprint arXiv:1804.09618* (2018).
- [22] Anna Choromanska et al. «The loss surfaces of multilayer networks». En: *Artificial intelligence and statistics*. PMLR. 2015, págs. 192-204.
- [23] Gareth James et al. «Statistical learning». En: *An introduction to statistical learning: With applications in Python*. Springer, 2023, págs. 15-67.
- [24] Yoshua Bengio, Ian Goodfellow, Aaron Courville et al. *Deep learning*. Vol. 1. MIT press Cambridge, MA, USA, 2017.
- [25] Aston Zhang et al. *Dive into deep learning*. Cambridge University Press, 2023.
- [26] Sanyuan Chen et al. «Wavlm: Large-scale self-supervised pre-training for full stack speech processing». En: *IEEE Journal of Selected Topics in Signal Processing* 16.6 (2022), págs. 1505-1518.
- [27] Wei-Ning Hsu et al. «Hubert: Self-supervised speech representation learning by masked prediction of hidden units». En: *IEEE/ACM transactions on audio, speech, and language processing* 29 (2021), págs. 3451-3460.